

OS 课程设计实验报告

2452293 冯昊天

Lab: Xv6 and Unix utilities 实验报告

1. Boot xv6 (easy)

1) 实验目的

熟悉 xv6 操作系统的编译和运行环境，了解 xv6 的基本结构和文件系统布局，为后续实验打下基础。

2) 实验步骤

1. 克隆 xv6 源码仓库: `git clone git://g.csail.mit.edu/xv6 - labs - 2025` `cd xv6-labs-2025`
2. 编译并运行 xv6: `$ make qemu`
3. 验证系统启动: 实际运行截图如下:

```

fht@LAPTOP-1TQ69S4D:/mnt/c/Users/冯昊天$ cd ~
fht@LAPTOP-1TQ69S4D:~$ cd xv6-*
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ make qemu
mv -f fs.img fs.img.bk
mkfs/mkfs fs.img README user/findtest.sh user/sixfive.txt user/_cat user/_echo user/_forktest use
r/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user
/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_s
leep user/_sixfive user/_memdump
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1031 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -glo
bal virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk
-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ls
.                1 1 1024
..               1 1 1024
README          2 2 2425
findtest.sh     2 3 91
sixfive.txt     2 4 19
cat             2 5 36504
echo            2 6 35320
forktest        2 7 17192
grep            2 8 39944
init            2 9 35792
kill            2 10 35264
ln              2 11 35072
ls              2 12 38616
mkdir           2 13 35320
rm              2 14 35312
sh              2 15 58352
stressfs        2 16 36184
usertests       2 17 186704
grind           2 18 51544
wc              2 19 37408
zombie          2 20 34664
logstress       2 21 37240
forphan         2 22 36136
dorphan         2 23 35592
sleep           2 24 35760
sixfive         2 25 38640
memdump         2 26 36696
console         3 27 0
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$

```

- 观察 qemu 启动过程，确认内核成功加载，显示 `xv6 kernel is booting`
- 等待 `init: starting sh` 出现，表明 shell 已就绪
- 输入 `ls` 命令查看文件系统中的文件列表

4. 退出 qemu:

- 按 `Ctrl-a x` 退出 qemu 模拟器

3) 实验中遇到的问题 and 解决办法

● 问题 1: 操作不熟悉系统无法正常启动

- 解决办法: 重新完整克隆 `xv6-labs-2025`，最终成功启动 `xv6` 系统

4) 实验心得

通过本次实验，成功搭建了 xv6 的编译和运行环境。`make qemu` 命令会自动完成编译、链接、生成文件系统镜像等一系列操作，最终启动 qemu 模拟器运行 xv6。系统启动后可以看到 shell 提示符，能够执行基本命令。这让我对操作系统的启动过程有了初步认识，熟悉了基本流程。

2. sleep (easy)

1) 实验目的

熟悉在 xv6 上编写 user program（用户级程序），掌握 `pause()` 系统调用的使用。

2) 实验步骤

1. 创建 `user/sleep.c`:

```
int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(2, "Usage: sleep <ticks>\n"); exit(1);
    }
    if (argv[1][0] == '-') {          // 负数
        fprintf(2, "sleep: invalid tick count\n"); exit(1);
    }
    for (int i = 0; argv[1][i]; i++) // 非数字
        if (argv[1][i] < '0' || argv[1][i] > '9') {
            fprintf(2, "sleep: invalid tick count\n"); exit(1);
        }
    pause(atoi(argv[1]));
    exit(0);
}
```
2. 在 Makefile 的 `UPROGS` 中添加 `$U/_sleep`。
3. 运行 `sleep 10` 验证。

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ sleep -1
sleep: invalid tick count
$ sleep a5
sleep: invalid tick count
$ sleep 10
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-util sleep
make: 'kernel/kernel' is up to date.
== Test sleep, no arguments == sleep, no arguments: OK (1.3s)
== Test sleep, returns == sleep, returns: OK (0.9s)
== Test sleep, makes syscall == sleep, makes syscall: OK (1.0s)
```

3) 实验中遇到的问题和解决办法

- 问题 1: 忘记在 Makefile 中添加 `_sleep`

- 解决办法: 在 Makefile 中添加 `_sleep` (实际可以只添加到 util 分支, 更加直观)

4) 实验心得

本次实验让我学会了如何在 xv6 上编写用户级程序。通过参考已有程序的结构, 我理解了命令行参数的处理方式和系统调用的使用。`pause()` 系统调用会使进程进入休眠状态, 直到被信号唤醒。同时, 我也学会了要将新程序添加到 Makefile 的 UPROGS 列表中, 才能使编译系统能够编译和链接新程序。

3. sixfive (moderate)

1) 实验目的

掌握文件操作系统调用 (`open`、`read`), 熟悉 C 字符串处理, 学习逐字符读取和解析文本文件。

2) 实验步骤

1. 创建 `user/sixfive.c`, 核心逻辑: `int is_separator(char c) {`
 `char separators[] = "-\r\t\n./,";` // 分隔符集合
 `return strchr(separators, c) != 0;`
 `}`
 `int is_multiple_of_5or6(int num) {`
 `return (num % 5 == 0) || (num % 6 == 0);`
 `}`
 `void process_file(char *filename) {`
 `int fd = open(filename, 0);` // 只读
 `char c; int num = 0, has_digit = 0;`
 `while (read(fd, &c, 1) > 0) {`
 `if (c >= '0' && c <= '9') {`
 `num = num * 10 + c - '0'; has_digit = 1;`
 `} else if (is_separator(c)) {`
 `if (has_digit && is_multiple_of_5or6(num))`
 `printf("%d\n", num);`
 `num = 0; has_digit = 0;`
 `}`
 `}`
 `if (has_digit && is_multiple_of_5or6(num))` // 文件末尾
 `printf("%d\n", num);`
 `close(fd);`
 `}`
2. 在 Makefile 的 `UPROGS` 中添加 `$U/_sixfive`。
3. 运行 `sixfive sixfive.txt` 验证。

```

fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ make qemu
mv -f fs.img fs.img.bk
mv: cannot stat 'fs.img': No such file or directory
make: [Makefile:288: newfs.img] Error 1 (ignored)
riscv64-unknown-elf-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -DSOL_UTIL -DLAB_UTIL -
MD -mmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-b
uiltin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin
-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fn
o-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -l. -fno-stack-
protector -fno-pie -no-pie -c -o user/sixfive.o user/sixfive.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_sixfive user/sixfive.o user/ulib.o
user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_sixfive > user/sixfive.asm
riscv64-unknown-elf-objdump -t user/_sixfive | sed '1,/SYMBOL TABLE/d; s/.*/ /; /`$/d' > user/sixfive.
sym
mkfs/mkfs fs.img README user/findtest.sh user/sixfive.txt user/_cat user/_echo user/_forktest user/_grep
user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests us
er/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sleep user/_sixfive u
ser/_memdump
mmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1031 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global vir
tio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=
x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sixfive sixfive.txt
5
100
18
6
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-util sixfive
make: 'kernel/kernel' is up to date.
== Test sixfive_test == sixfive_test: OK (1.0s)
(Old xv6.out.sixfive_test failure log removed)
== Test sixfive_readme == sixfive_readme: OK (1.3s)
(Old xv6.out.sixfive_readme failure log removed)
== Test sixfive_all == sixfive_all: OK (1.0s)

```

3) 实验中遇到的问题和解决办法

- 问题 1：没有注意到分隔符的干扰

- 解决办法：用 `strchr(separators, c)` 判断当前字符是否为 `-`、`\r`、`\t`、`\n`、`./`、`/`、`,` 中的任意一个。遇到分隔符时才检查累积的数字是否为 5 或 6 的倍数并输出，然后重置 `num` 和 `has_digit`。如果没有分隔符判断，会把相邻的多个数字连成一个错误的大数。

4) 实验心得

本次实验加深了我对文件系统调用和 C 字符串处理的理解。通过逐字符读取文件并解析数字序列，我学会了如何处理文本数据。`strchr()` 函数是一个非常有用的工具，可以快速判断字符是否属于某个字符集。

4. memdump (easy)

1) 实验目的

深入理解 C 指针的使用，掌握不同数据类型的内存布局，学习格式化输出内存内容。

2) 实验步骤

1. 实现 `memdump(char *fmt, char *data)` 函数，用 `char *p` 遍历 `data`，根据 `fmt` 中的格式字符读取并推进指针： `void memdump(char *fmt, char *data) {`

```
char *p = data;
for (int i = 0; fmt[i]; i++) {
    switch (fmt[i]) {
        case 'i': printf("%d\n", *(int*)p);    p += 4; break;
        case 'p': printf("%lx\n", *(unsigned long*)p); p += 8; break;
        case 'h': printf("%d\n", *(short*)p);    p += 2; break;
        case 'c': printf("%c\n", *p);            p += 1; break;
        case 's': printf("%s\n", *(char**)p);    p += 8; break; // 指针解引用
        case 'S': printf("%s\n", p);            break;        // 剩余全部
    }
}
```

- `i`: 4 字节 `int`; `p`: 8 字节十六进制指针; `h`: 2 字节 `short`; `c`: 1 字节 `char`
- `s`: 8 字节是指针, `*(char**)p` 解引用得到字符串地址; `S`: 从 `p` 位置开始打印剩余全部

2. 运行 `memdump` 验证内置的 5 个 Example。

```

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ memdump
Example 1:
61810
2025
Example 2:
a string
Example 3:
another
Example 4:
B70
1819438967
100
z
xyzzzy
Example 5:
hello
w
o
r
l
d
$ echo deadc0de | memdump hhcccc
25956
25697
c
0
d
e
$ echo deadc0de | memdump p
64616564
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-util memdump
make: 'kernel/kernel' is up to date.
== Test memdump, examples == memdump, examples: OK (0.9s)
(Old xv6.out.memdump_examples failure log removed)
== Test memdump, format ii, S, p == memdump, format ii, S, p: OK (0.9s)

```

3) 实验中遇到的问题和解决办法

- 问题 1：不同格式字符的字节偏移控制

- 解决办法：不同格式字符消费的字节数不同（i 为 4 字节、p 为 8 字节、h 为 2 字节、c 为 1 字节、s 为 8 字节），使用一个 char *p 指针遍历 data，每处理一个格式字符就按对应字节数推进指针（p += 4/p += 8/p += 2/p += 1），确保后续读取的偏移正确。

- 问题 2：s 格式是指针解引用而非直接读取

- 解决办法：s 格式中，data 里存储的 8 字节是一个指向字符串的指针，需要 *(char**)p 解引用得到字符串地址后再 printf("%s\n", ...)。S 格式则直接从 p 位置开始打印剩余数据为字符串，不推进指针。两者区别在于 s 读取一个指针、S 读取内联数据。

4) 实验心得

本次实验让我深入理解了 C 指针和内存布局。通过实现 `memdump()` 函数，我学会了如何根据格式字符串解析内存中的不同数据类型。需要特别注意数据的字节序问题，以及指针的正确类型转换。不同数据类型占用不同的字节数，在处理内存时需要精确控制指针的移动。这个实验对于理解计算机系统的底层工作原理非常有帮助。

5. find (moderate)

1) 实验目的

掌握目录遍历和路径处理，理解文件系统调用 (`open`、`read`、`fstat`)，学习递归算法在文件系统操作中的应用。

2) 实验步骤

```
1. 创建 user/find.c，核心递归函数：void find(char *path, char *target) {
    int fd; struct dirent de; struct stat st; char buf[512], *p;
    if ((fd = open(path, 0)) < 0) { fprintf(2, "find: cannot open %s\n", path); return; }
    if (fstat(fd, &st) < 0) { close(fd); return; }
    if (st.type != T_DIR) {          // 非目录，直接比较文件名
        // 提取文件名后与 target 比较
        if (strcmp(filename, target) == 0) printf("%s\n", path);
        close(fd); return;
    }
    while (read(fd, &de, sizeof(de)) == sizeof(de)) {
        if (de.inum == 0 || strcmp(de.name, ".") == 0
            || strcmp(de.name, "..") == 0) continue; // 跳过 . 和 ..
        // 拼接子路径：path + "/" + de.name
        *p = '/'; p++; memmove(p, de.name, DIRSIZ); p[DIRSIZ] = 0;
        find(buf, target);          // 递归
    }
}
```

```
close(fd);
}
```

2. 在 Makefile 的 UPROGS 中添加 `$U/_find`。

3. 运行 `find . ls` 验证。

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ mkdir a/aa
$ echo > a/aa/b
$ find . b
./b
./a/b
./a/aa/b
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-util find
make: 'kernel/kernel' is up to date.
== Test find, in current directory == find, in current directory: OK (0.8s)
(Old xv6.out.find_curdir failure log removed)
== Test find, in sub-directory == find, in sub-directory: OK (0.9s)
(Old xv6.out.find_subdir failure log removed)
== Test find, recursive == find, recursive: OK (1.3s)
(Old xv6.out.find_recursive failure log removed)
== Test exec, recursive find == exec, recursive find: FAIL (0.8s)
Number of appearances of 'hello'
got:
0
expected:
3
QEMU output saved to xv6.out.test find sh
```

3) 实验中遇到的问题 and 解决办法

- 问题 1: 递归遍历陷入死循环

- 解决办法: 最初没有跳过 `.` 和 `..` 目录项, 导致 `find` 不断进入当前目录和父目录形成无限递归。在 `read(fd, &de, sizeof(de))` 读取目录项后用 `strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0` 判断, 遇到时直接 `continue` 跳过。

- 问题 2: 路径拼接的内存管理

- 解决办法: 拼接子目录路径时需要构造 父路径/子名 形式的字符串。使用局部缓冲区 `char buf[512]`, 先 `memmove(buf, path, strlen(path))`, 再在末尾添加 `/` 和 `de.name`, 递归返回后缓冲区自动释放, 无需手动管理堆内存。

- 问题 3: 文件类型判断错误

- **解决办法**：仅靠文件名无法区分普通文件和目录，导致对普通文件调用 `read()` 读取目录项时报错。对每个路径先 `open()` 再 `fstat(fd, &st)`，通过 `st.type == T_DIR` 判断是否为目录，仅对目录递归遍历。

4) 实验心得

本次实验让我学会了如何遍历文件系统目录。通过参考 `ls.c` 的实现，我理解了目录项的结构和读取方式。递归算法在目录遍历时非常重要，可以自动处理任意深度的子目录。同时，我也学会了如何使用 `fstat()` 判断文件类型，区分普通文件和目录。路径拼接也是一个关键点，需要注意字符串的拼接和内存管理。

6. exec (moderate)

1) 实验目的

掌握进程创建和执行系统调用 (`fork`、`exec`、`wait`)，理解进程管理机制，学习如何在程序中执行外部命令。

2) 实验步骤

1. 在 `user/find.c` 中添加 `-exec` 支持，全局变量保存命令：`char* exec_cmd = 0;`
`char* exec_args[MAXARG];`在 `main` 中解析 `-exec` 参数：`exec_cmd = argv[exec_pos + 1];` // `-exec` 后的第一个参数是命令
`int arg_idx = 0;`
`for (int i = exec_pos + 2; i < argc; i++)` // 后续参数存入 `exec_args`
 `exec_args[arg_idx++] = argv[i];`
`exec_args[arg_idx] = 0;` // null 结尾
2. 在 `find()` 中匹配到文件后执行命令：`if (exec_cmd != 0) {`
 `int pid = fork();`
 `if (pid == 0) {` // 子进程
 `char* argv[MAXARG]; int i = 0;`
 `argv[i++] = exec_cmd;` // 命令名

```

for (int j = 0; exec_args[j]; j++) // 附加参数
    argv[i++] = exec_args[j];
argv[i++] = path;                // 文件路径作为最后参数
argv[i] = 0;
exec(exec_cmd, argv);            // 执行命令
fprintf(2, "find: exec failed\n"); exit(1);
} else {                          // 父进程
    wait(0);                      // 等待子进程完成
}
} else {
    printf("%s\n", path);         // 无 -exec, 直接打印
}
}

```

3. 运行 `find . -exec grep hello` 验证。

```

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ find . b -exec echo hi
hi ./b
hi ./a/b
$ sh < findtest.sh
$ mkdir: a failed to create
$ $ $ $ hello
hello
hello
$ $ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-util find
make: 'kernel/kernel' is up to date.
== Test find, in current directory == find, in current directory: OK (1.3s)
== Test find, in sub-directory == find, in sub-directory: OK (0.9s)
== Test find, recursive == find, recursive: OK (1.2s)
== Test exec, recursive find == exec, recursive find: OK (1.0s)
(Old xv6.out.test_find_sh failure log removed)

```

3) 实验中遇到的问题解决办法

● 问题 1: `-exec` 参数解析困难

- 解决办法: 命令行中 `-exec` 后跟要执行的命令名, 再之后才是 `find` 的路径参数。扫描 `argv` 数组定位 `-exec` 的位置 `exec_pos`, `argv[exec_pos+1]` 是命令名存入全局 `exec_cmd`, `argv[exec_pos+2..]` 存入 `exec_args[]` 数组并以 0 结尾。`find` 的搜索路径参数仍在 `-exec` 之前, 两类参数不混淆。

● 问题 2: `fork` 后 `wait` 的顺序控制

- 解决办法: 最初忘记在父进程中调用 `wait()`, 导致子进程成为僵尸进程。在每次 `fork()` 后于父进程分支 (`else`) 立即调用 `wait(0)`, 等待该子进程执行完毕再继续查找下一个文件, 保证输出顺序与查找顺序一致。

4) 实验心得

本次实验让我深入理解了进程管理机制。`fork()` 会创建一个新进程，`exec()` 会替换进程的代码段，`wait()` 会等待子进程结束。通过组合使用这三个系统调用，可以实现进程的创建和外部命令的执行。这个实验让我明白了 UNIX 系统中命令执行的底层原理，也理解了 shell 的工作机制。同时，参数解析和命令构造也是需要仔细处理的部分。

Lab: System Calls

1. Using gdb (easy)

1) 实验目的

本实验旨在学习使用 GDB 调试 xv6 内核，掌握内核调试的基本方法，包括设置断点、查看堆栈回溯、打印进程结构和寄存器值等。通过调试实践，深入理解系统调用的执行流程和内核机制。

2) 实验步骤

1. 启动 GDB 调试环境：make qemu-gdb 在另一个终端窗口启动 GDB：gdb-multiarch

2. 设置断点并查看堆栈：(gdb) b syscall

Breakpoint 1 at 0x80001cca: file kernel/syscall.c, line 133.

(gdb) c

Continuing.

(gdb) layout src

(gdb) backtrace

3. 查看进程结构：

执行 n 命令单步执行后：(gdb) p /x *p

4. 查看系统调用号和状态寄存器: (gdb) p /x p->trapframe->a7

(gdb) p /x \$sstatus

5. 定位内核 panic 原因:

修改 `syscall.c` 中的代码引入错误: `num = *(int *)0`; 运行后观察 panic 信息, 在 `kernel/kernel.asm` 中查找对应指令: `grep -n "80001cda" kernel/kernel.asm`

6. 在 GDB 中查看崩溃指令: (gdb) b *0x80001cda

(gdb) layout asm

(gdb) c 运行截图如下:

```
(gdb) b syscall
Breakpoint 1 at 0x80001cca: file kernel/syscall.c, line 133.
(gdb) c
Continuing.

kernel/syscall.c
127 [SYS_mkdir] sys_mkdir,
128 [SYS_close] sys_close,
129 ];
130
131 void
132 syscall(void)
133 {
134     int num;
135     struct proc *p = myproc();
136
137     num = p->trapframe->a7;
138     if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
139         // Use num to lookup the system call function for num, call it,
140         // and store its return value in p->trapframe->a0
141         p->trapframe->a0 = syscalls[num]();
142     } else {
143         printf("%d %s: unknown sys call %d\n",
144             p->pid, p->name, num);
145         p->trapframe->a0 = -1;
146     }
147 }

remote Thread 1.2 (src) In: syscall L137 PC: 0x80001cdc
(gdb) backtrace
#0  syscall () at kernel/syscall.c:133
#1  0x0000000080001a90 in usertrap () at kernel/trap.c:68
#2  0x00000003ffffff09c in ?? ()
(gdb) n
(gdb) n
(gdb) p /x p->trapframe->a7
$1 = 0x5
(gdb) p /x $sstatus
$2 = void
(gdb) p /x $sstatus
$3 = 0x200000022
(gdb) -
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ grep -n "80001cda" kernel/kernel.asm
4406: 80001cda: 00002683 lw a3,0(zero) # 0 <_entry-0x80000000>
```

```
0x80001cd0 <syscall+12> jal 0x80000d70 <myproc>
B> 0x80001cda <syscall+16> mv s1,a0
0x80001cdc <syscall+18> ld s2,88(a0)
0x80001ce0 <syscall+22> ld a5,168(s2)
0x80001ce4 <syscall+26> sext.w a3,a5
0x80001ce8 <syscall+30> addiw a5,a5,-1

remote Thread 1.2 (asm) In: syscall L135 PC: 0
(gdb) c
Continuing.
[Switching to Thread 1.2]

Thread 2 hit Breakpoint 1, 0x0000000080001cda in syscall () at kernel/syscall.c:135
(gdb) p p->name
x value has been optimized out
(gdb) p ((struct proc *)$a0)->name
$1 = "init", '\000' <repeats 11 times>
(gdb) p ((struct proc *)$a0)->pid
x A syntax error in expression, near the end of `((struct proc *)$a0)->pid'.
(gdb) p ((struct proc *)$a0)->pid
$2 = 1
(gdb)
```

3) 实验中遇到的问题和解决办法

- 问题 1：断点设置后无法命中
 - 解决办法：确保 QEMU 和 GDB 连接正常，使用正确的断点地址。
- 问题 2：打印进程名称时提示 "optimized out"
 - 解决办法：使用指针转换方式：`p ((struct proc *)$a0)->name`。

4) 实验心得

通过 GDB 调试实践，我深入理解了系统调用的执行流程：用户程序通过 `ecall` 指令触发陷入，内核在 `usertrap` 中处理，最终调用 `syscall` 函数分发到具体的系统调用处理函数。调试过程中，我学会了如何利用堆栈回溯追踪代码执行路径，如何查看和解释特权寄存器的值，以及如何定位内核崩溃的根本原因。

2. Sandbox a command (moderate)

1) 实验目的

本实验要求实现一个新的 `interpose` 系统调用，用于限制进程可调用的系统调用。通过位掩码指定禁止的系统调用，子进程继承父进程的限制，从而实现进程沙箱功能。

2) 实验步骤

1. 添加系统调用声明:

- 在 `user/user.h` 中添加 `interpose` 原型: `int interpose(int mask, char* path);`
- 在 `user/usys.pl` 中添加系统调用 stub: `entry("interpose");`
- 在 `kernel/syscall.h` 中添加系统调用号: `#define SYS_interpose 22`
- 在 `kernel/syscall.c` 中添加函数原型声明，并在 `syscalls[]` 数组中加入对应项: `extern uint64 sys_interpose(void);`
- ...
- `[SYS_interpose] sys_interpose,`

2. 在进程结构中添加字段:

在 `kernel/proc.h` 的 `struct proc` 中添加掩码与允许路径: `int mask;`
`char allowed_path[MAXPATH];`

3. 实现内核系统调用处理函数 `sys_interpose`:

在 `kernel/sysproc.c` 中实现，从用户态取出 `mask` 与 `path`，写入当前进程: `uint64 sys_interpose(void)`

```
{
    int mask;
    char path[64];
    struct proc* p = myproc();
    argint(0, &mask);
    argstr(1, path, sizeof(path));
    p->mask = mask;
    safestrcpy(p->allowed_path, path, MAXPATH);
    return 0;
}
```

4. 修改 `kfork` 让子进程继承掩码:

在 `kernel/proc.c` 的 `kfork` 中复制父进程的 `mask` 与 `allowed_path`: `np->mask = p->mask;`
`safestrcpy(np->allowed_path, p->allowed_path, MAXPATH);`

5. 修改 `syscall` 函数进行掩码检查:

在 `kernel/syscall.c` 的 `syscall` 中，读取调用号 `num` 后先判断 `p->mask & (1 << num)`:

- 若命中且为 `SYS_open/SYS_exec`，不直接拒绝，而是调用 `syscalls[num]()` 本身，交由对应函数内部按路径放行;
- 若命中且为其他系统调用，直接 `p->trapframe->a0 = -1` 返回;

- 未命中则按原逻辑分发。

6. 修改 `sys_open/sys_exec` 实现路径豁免：

在 `kernel/sysfile.c` 中，`sys_open` 和 `sys_exec` 取出路径参数后，若 `p->mask & (1 << SYS_open)` 命中，则逐字符比较 `path` 与 `p->allowed_path`，不匹配则返回 -1。

7. 在 `Makefile` 中添加用户程序：

8. 测试验证：\$ `sandbox 32768 - cat README`

`cat: cannot open README` 运行截图如下：

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ sandbox 32768 - cat README
cat: cannot open README
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-syscall sandbox_mask
make: 'kernel/kernel' is up to date.
== Test sandbox_mask == sandbox_mask: OK (1.3s)
```

3) 实验中遇到的问题 and 解决办法

● 问题 1：新增系统调用需要改动多处文件，容易遗漏声明

- 解决办法：`interpose` 系统调用需要同时修改 `user/user.h`（用户态原型）、`user/usys.pl`（生成用户态 stub）、`kernel/syscall.h`（调用号 `SYS_interpose`）、`kernel/syscall.c`（函数原型声明 + `syscalls[]` 数组项）以及 `kernel/sysproc.c`（实现体 `sys_interpose`）。任一处遗漏都会导致链接报错或调用号无法分发，逐步对照检查后补全全部五处声明。

● 问题 2：fork 后子进程的 `mask` 为默认值 0，逃逸沙箱

- 解决办法：`proc.h` 中新增的 `mask` 和 `allowed_path` 字段在 `allocproc()` 里不会自动初始化，`kfork()` 复制进程结构时默认只复制内核栈和 `trapframe`。若不显式复制，子进程 `mask` 为 0（无任何限制），沙箱形同虚设。在 `kernel/proc.c` 的 `kfork()` 中补上 `np->mask = p->mask;` 和 `safestrcpy(np->allowed_path, p->allowed_path, MAXPATH);`，让子进程继承完整的掩码与允许路径。

● 问题 3：掩码命中的 `open/exec` 不能直接拒绝，需要路径例外

- 解决办法：`syscall()` 中对掩码命中的系统调用本应直接 `a0 = -1` 返回，但 `SYS_open/SYS_exec` 存在路径豁免：若访问的路径等于 `allowed_path` 则应放行。最初把拒绝逻辑写死在 `syscall()` 开头，导致即使路径匹配也被拒绝。改为在 `syscall()` 中对掩码命中的 `open/exec` 不直接拒绝，而是调用

`syscalls[num]()` 本身，由 `sys_open/sys_exec` 内部自行比对 `p->allowed_path` 与请求路径，只有路径不匹配时才返回 -1。

- **问题 4: `sys_open/sys_exec` 中路径匹配逻辑的实现**

- **解决办法:** 在 `kernel/sysfile.c` 的 `sys_open` 和 `sys_exec` 中，先用 `argstr()` 取出用户传入的路径，再判断 `p->mask & (1 << SYS_open)` 是否命中；命中则用 `while` 循环逐字符比较 `path` 与 `p->allowed_path`，若任一字符不等则判为不匹配、返回 -1。这里没有直接用 `strcmp()`，因为需要在比较的同时判断是否到达两者末尾，手动逐字符比较更便于控制匹配语义。

4) 实验心得

实现 `interpose` 系统调用让我对 xv6 的系统调用机制有了深入理解。系统调用的完整流程包括：用户空间 `stub` 通过 `ecall` 进入内核，内核通过 `trapframe` 获取系统调用号和参数，然后通过函数指针表分发到具体的处理函数。进程沙箱机制的实现涉及到进程结构的扩展、系统调用路径的拦截以及父子进程间状态的继承。这部分实验锻炼了我对内核代码的修改和调试能力。

3. Sandbox with allowed pathnames (easy)

1) 实验目的

扩展沙箱功能，允许基于路径名的过滤机制。当 `open` 或 `exec` 系统调用被禁止时，如果访问的路径名与允许的路径名匹配，则允许执行该系统调用。

2) 实验步骤

本实验在上一节 `Sandbox a command` 基础上扩展：`syscall()` 已对掩码命中的 `open/exec` 转交给 `sys_open/sys_exec` 自行处理，本节需要在这两个函数内部加入路径豁免逻辑。

1. 在 `sys_open` 中加入路径检查:

在 `kernel/sysfile.c` 的 `sys_open` 中, `argstr(0, path, MAXPATH)` 取出路径后、`begin_op()` 之前插入: `if(p->mask & (1 << SYS_open)) {`

```
int same = 1;
int i = 0;
while (path[i] && p->allowed_path[i] && path[i] == p->allowed_path[i]) {
    i++;
}
if (path[i] != p->allowed_path[i]) {
    same = 0;
}
if (!same) {
    return -1;
}
}
```

2. 在 `sys_exec` 中加入路径检查:

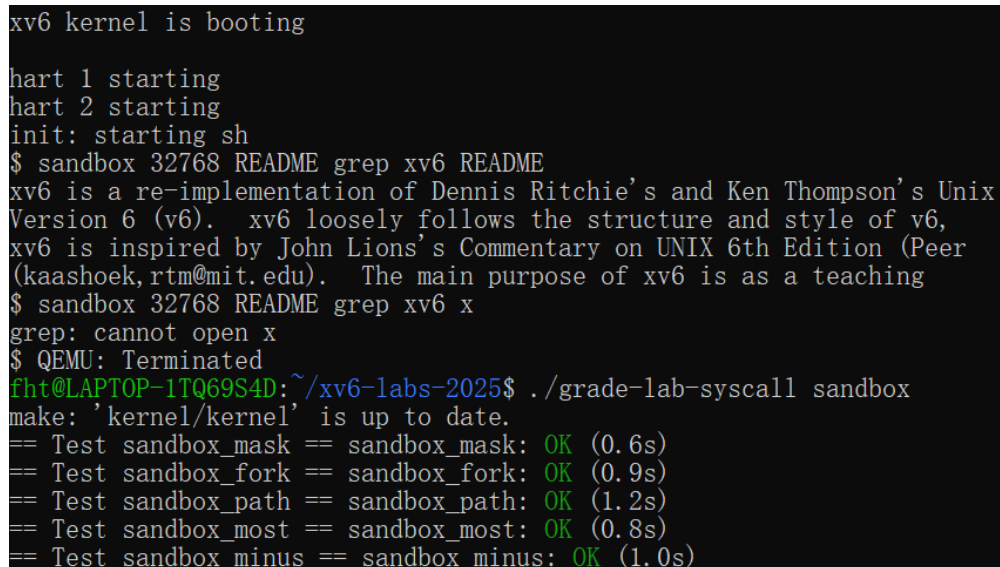
在 `kernel/sysfile.c` 的 `sys_exec` 中, `argstr(0, path, MAXPATH)` 取出路径后、解析 `uargv` 之前插入与 `sys_open` 相同的掩码判断与逐字符比较逻辑 (把 `SYS_open` 换成 `SYS_exec`)。

3. 测试验证: `sandbox32768READMEgrep xv6README xv6isare -`

`implementationofDennisRitchie'sandKenThompson'sUnixVersion6(v6).xv6looselyfollowsthestr`

`sandbox 32768 README grep xv6 x`

`grep: cannot open x` 运行截图如下:



```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ sandbox 32768 README grep xv6 README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6). xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu). The main purpose of xv6 is as a teaching
$ sandbox 32768 README grep xv6 x
grep: cannot open x
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-syscall sandbox
make: 'kernel/kernel' is up to date.
== Test sandbox_mask == sandbox_mask: OK (0.6s)
== Test sandbox_fork == sandbox_fork: OK (0.9s)
== Test sandbox_path == sandbox_path: OK (1.2s)
== Test sandbox_most == sandbox_most: OK (0.8s)
== Test sandbox_minus == sandbox_minus: OK (1.0s)
```

3) 实验中遇到的问题和解决办法

- 问题 1: 路径检查应该放在 `syscall()` 还是 `sys_open/sys_exec` 中

- **解决办法**: 最初想在 `syscall()` 里直接取路径参数比较, 但 `syscall()` 只拿到调用号 `num`, 取路径还要再 `argstr`, 且 `exec` 的参数布局与 `open` 不同, 逻辑会很乱。最终把检查下沉到 `sys_open/sys_exec` 内部——这两个函数本来就用 `argstr(0, path, MAXPATH)` 取出了路径, 直接在取路径后比较即可, `syscall()` 只负责把掩码命中的 `open/exec` 转交给它们。
- **问题 2: 逐字符比较何时停止、如何判定不匹配**
 - **解决办法**: 用 `while (path[i] && p->allowed_path[i] && path[i] == p->allowed_path[i])` 循环, 任一字符串到 `\0` 或字符不等就退出。退出后再判断 `path[i] != p->allowed_path[i]`: 若两者都到末尾 (都为 `\0`) 则完全相等、放行; 若只有一个到末尾或字符不等, 则 `same=0` 拒绝。这样能正确区分"完全匹配"与"仅前缀匹配"。
- **问题 3: `allowed_path` 为前缀时被误判放行**
 - **解决办法**: 如果只比较到 `allowed_path` 末尾就停止, 会把 `README` 和 `README.txt` 都判为匹配。通过循环退出后的 `path[i] != p->allowed_path[i]` 判断, 要求两者必须同时到达 `\0` 才算匹配, 避免了前缀误匹配。

4) 实验心得

路径名过滤功能的实现涉及到系统调用参数的动态获取和条件判断。在实现过程中, 我需要深入理解不同系统调用的参数结构, 特别是 `open` 和 `exec` 如何接收路径名参数。通过 `argint` 和 `argstr` 等辅助函数, 可以方便地从用户空间获取系统调用参数。这个实验让我认识到系统调用拦截和过滤的实现原理, 对于理解安全机制和沙箱技术有很大帮助。

4. Attack xv6 (moderate)

1) 实验目的

本实验利用 `xv6` 中内存未清零的漏洞, 通过 `sbrk` 系统调用分配内存, 获取前一个进程释放的内存页中残留的秘密数据。这展示了即使不直接影响正确性的 bug 也可能被利用来破坏安全性。

2) 实验步骤

1. 分析漏洞原理与 `secret.c` 的布局:

- 实验移除了 `kernel/vm.c` 中 `uvmmalloc()` 的 `memset(mem, 0, sz)` 和 `kernel/kalloc.c` 中的 `memset`, 新分配的物理页不再清零, 保留了上一个使用者的内容。
- `user/secret.c` 把 "This may help." 写入全局 `data[0]`, 把命令行传入的秘密写入 `data[16]`, 进程退出后这些页被释放但内容残留。
- `data` 大小为 `8*4096` (8 页), 秘密字符串位于标记后 16 字节处。

2. 编写攻击程序 `user/attack.c`: `#define PGSIZE 4096`

```
int is_secret_char(char c) {
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9');
}

int main(int argc, char *argv[]) {
    int pages = 64;
    char* start = sbrk(0);
    for (int i = 0; i < pages; i++) {
        if (sbrk(PGSIZE) == (char*)-1) exit(1);
    }
    for (int i = 0; i < pages * PGSIZE - 20; i++) {
        if (start[i] == 'T' && start[i+1] == 'h' && /* ... */ start[i+13] == '.') {
            char* secret = start + i + 16;
            int len = 0;
            while (len < 100 && is_secret_char(secret[len])) len++;
            if (len >= 1 && secret[len] == '\0') {
                printf("%s\n", secret);
                exit(0);
            }
        }
    }
    exit(0);
}
```

- 先用 `sbrk(0)` 记录起始地址 `start`, 再循环调用 `sbrk(PGSIZE)` 分配 64 页, 覆盖 `secret` 释放的物理页。
- 逐字节扫描 `start[0 .. pages*PGSIZE-20]`, 寻找标记 "This may help." (逐字符比较 `T h i s m a y h e l p .`)。
- 找到标记后, 秘密在 `start + i + 16` 处; 用 `is_secret_char()` 统计连续字母数字长度, 要求 `len >= 1` 且 `secret[len] == '\0'` (以 null 结尾), 通过后 `printf` 输出。

3. 测试攻击: `secretxyzyzy attack`

xyzzzy 运行截图如下:

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ secret xyzzzyx
$ attack
xyzzzyx
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-syscall attack
make: 'kernel/kernel' is up to date.
== Test attack == attack: OK (1.0s)
(Old xv6.out.attack failure log removed)
```

3) 实验中遇到的问题和解决办法

- 问题 1: 如何在残留内存中定位秘密的位置

- 解决办法: 直接遍历整个分配区域找可打印字符串不可行, 因为残留内容里有很多非秘密数据。利用 `secret.c` 写入的固定标记 "This may help." 作为定位锚点, 秘密就在标记后 16 字节处 (`data[16]`)。逐字符匹配这 14 个标记字符即可精确定位。

- 问题 2: 分配的页未命中 `secret` 释放的物理页

- 解决办法: 物理页分配由 `kalloc` 从空闲链表取出, 顺序不确定, 少量分配可能拿不到 `secret` 用过的页。把分配页数从最初的 10 页增加到 64 页 (`pages = 64`), 用足够多的分配提高命中 `secret` 残留页的概率。

- 问题 3: 输出内容混入乱码或非秘密数据

- 解决办法: 残留内存里除秘密外还有其他残留字节, 直接 `printf` 会打印乱码。加入 `is_secret_char()` 过滤——只接受字母数字字符, 并要求秘密字符串以 `\0` 结尾 (`secret[len] == '\0'`), 确保输出的是完整、干净的秘密字符串而非垃圾数据。

- 问题 4: 扫描越界访问导致段错误

- 解决办法: 标记匹配需要向后读 14 字节、秘密校验最多读 100 字节, 若在区域末尾附近匹配到部分标记会越界。把扫描上界设为 `pages * PGSIZE - 20`, 给标记和秘密读取留出足够余量, 避免访问未分配区域。

4) 实验心得

这个实验生动地展示了内存安全的重要性。即使是看似无关紧要的初始化操作, 也可

能导致严重的安全漏洞。攻击者可以通过分配内存来获取前一个进程的敏感数据，这在真实系统中可能导致密码泄露、密钥窃取等严重后果。实验让我深刻理解了为什么现代操作系统和编程语言如此重视内存安全，以及为什么需要内存清零、地址空间随机化等安全机制来保护系统。

Lab: page tables

1. Inspect a user-process page table (easy)

1) 实验目的

本实验的目的是帮助理解 RISC-V 页表结构，通过分析用户进程的页表输出，掌握页表项（PTE）中各字段的含义，包括虚拟地址、页表项内容、物理地址和权限位，并理解用户进程地址空间中不同页面的用途。

2) 实验步骤

1. 切换到 `pgtbl` 分支：`git fetch git checkout pgtbl`
`$ make clean`
2. 运行 `make qemu` 启动 xv6 虚拟机。
3. 执行用户程序 `pgtbltest`，观察 `print_pgtbl` 函数的输出。
4. 分析输出中前 10 个和后 10 个页表项，解释每个字段的含义和权限位。

实验运行截图：


```

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x2000 pte 0x21FC7817 pa 0x87F1E000 perm 0x17
va 0x3000 pte 0x21FC7407 pa 0x87F1D000 perm 0x7
va 0x4000 pte 0x21FC70D7 pa 0x87F1C000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFE000 pte 0x21FD08C7 pa 0x87F42000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
usertrap(): unexpected scause 0xd pid=3
          sepc=0x7de stval=0x3fffffd000

```

3) 实验中遇到的问题和解决办法

- 问题 1: 权限位含义

- 解决办法: 参考 xv6 源码中 `kernel/riscv.h` 文件中的 PTE 宏定义, 明确每个位的含义:

- 问题 2: 高地址区域虚拟地址格式不直观

- 解决办法: 查阅 xv6 内存布局图, 了解用户地址空间的高地址区域主要用于 Trapframe、Trampoline 和 USYSCALL 等特殊页面。

4) 实验心得

通过本次实验，我深入理解了 RISC-V Sv39 分页机制的基本原理。页表项不仅包含物理地址，还包含丰富的权限控制位，这些权限位共同构成了内存安全的基础。用户进程的地址空间是精心设计的，从低地址的代码段和数据段，到高地址的栈、Trapframe 和 Trampoline，每个区域都有其特定的用途和权限设置。特别是 U=0 的页面，确保了内核数据对用户程序的隔离，这是操作系统安全性的重要保障。

2. Speed up system calls (easy)

1) 实验目的

本实验的目的是实现一种优化系统调用的技术：通过在内核和用户空间之间共享一个只读页面，让用户程序可以直接读取某些系统信息，从而避免系统调用带来的内核切换开销。具体实现 `getpid()` 系统调用的优化。

2) 实验步骤

1. 在 `kernel/proc.h` 的 `struct proc` 中添加字段： `struct usyscall* usyscall;`
2. 在 `allocproc()` 中分配并初始化 `usyscall` 页面：
参考 `trapframe` 的分配方式，用 `kalloc()` 分配一页，并把进程 PID 写入： `if ((p->usyscall = (struct usyscall*)kalloc()) == 0) {
 freeproc(p); release(&p->lock); return 0;
}
...
np->usyscall->pid = np->pid;`
3. 在 `proc_pagetable()` 中将 `usyscall` 页面映射到 `USYSCALL` 虚拟地址： `if (mappages(pagetable, USYSCALL, PGSIZE,
 (uint64)p->usyscall, PTE_R | PTE_U) < 0) {
 uvmunmap(pagetable, USYSCALL, 1, 0);
 freewalk(pagetable); return 0;
}` 权限设为 `PTE_R | PTE_U`（只读 + 用户可访问）。
4. 在 `freeproc()` 中释放 `usyscall` 页面： `if (p->usyscall) {
 kfree((void*)p->usyscall);
 p->usyscall = 0;`

```
}
```

5. 运行 `pgtbltest`, 验证 `ugetpid_test` 通过。

实验运行截图:

```
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
print_kpgtbl: OK
superpg_fork starting
pgtbltest: superpg_fork failed: pte different, pid=4
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-pgtbl ugetpid
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:22: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. 0x0000000000000000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:23: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. \. \. 0x0000000000000000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:24: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. \. \. \. \. 0x0000000000000000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:25: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. \. \. \. \. 0x0000000000000100',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:26: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. \. \. \. \. 0x0000000000000200',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:27: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. \. \. \. \. 0x0000000000000300',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:28: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. (0xffffffffc0000000|0x0000003fc0000000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:29: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. \. \. (0xffffffffffe00000|0x0000003fffe00000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:30: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. \. \. \. \. (0xffffffffffffd000|0x0000003fffffd000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:31: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. \. \. \. \. (0xfffffffffffffe000|0x0000003fffffe000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:32: SyntaxWarning: "
Such sequences will not work in the future. Did you mean "\\."
'^' \. \. \. \. \. \. (0xfffffffffffff000|0x0000003fffff000)',
make: 'kernel/kernel' is up to date.
== Test pgtbltest == (0.8s)
== Test pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$
```

3) 实验中遇到的问题和解决办法

- 问题 1: 运行 `ugetpid_test` 出现 `usertrap(): unexpected scause 0xd` 错误
 - 解决办法: `scause 0xd` 是 load 页面缺失, 说明用户态读 `USYSCALL` 地址时页表里没有合法映射。最初 `mappages` 的权限只设了 `PTE_R`, 缺少 `PTE_U`, 导致用户态访问被拒绝。把权限改为 `PTE_R | PTE_U` 后, 用户程序可以直接只读访问该共享页面, 无需陷入内核。
- 问题 2: `freeproc` 中忘记释放 `usyscall` 页面导致内存泄漏
 - 解决办法: `allocproc()` 中 `kalloc()` 分配的 `usyscall` 页面, 如果 `freeproc()` 不显式 `kfree`, 进程退出后该物理页永远不会归还空闲链表。在 `freeproc()` 中补上 `if (p->usyscall) { kfree((void*)p->usyscall); p->usyscall = 0; }`, 与 `trapframe` 的释放方式一致。
- 问题 3: `proc_pagetable` 映射失败时未清理已映射的 `USYSCALL` 项
 - 解决办法: `proc_pagetable` 中先映射 `trapframe` 再映射 `USYSCALL`, 如果 `USYSCALL` 映射失败需要回滚, 但 `trapframe` 已经映射进去了。在失败路径中调用 `uvmunmap(pagetable, USYSCALL, 1, 0)` 清理可能残留的映射项, 再 `freewalk` 释放页表本身, 避免 `freewalk` 遇到 leaf PTE 触发 panic。

4) 实验心得

共享页面是一种非常巧妙的优化技术, 它将某些只读的系统信息直接暴露给用户空间, 避免了系统调用的开销。这种技术适用于那些返回值不经常变化且不需要复杂计算的系统调用。除了 `getpid()`, 像 `uptime()` 这样的系统调用也可以采用类似的优化方式, 将系统时钟计数存放在共享页面中。但对于 `read()`、`write()` 等需要执行内核操作或修改系统状态的系统调用, 则无法采用这种方式。

3. Print a page table (easy)

1) 实验目的

本实验的目的是实现一个页表打印函数 `vmprint()`，帮助可视化 RISC-V 三级页表的结构，理解页表的层次关系，并为后续调试提供工具。

2) 实验步骤

1. 在 `kernel/vm.c` 中实现 `vmprint()`：

打印标题行后调用递归辅助函数：`void vmprint(pagetable_t pagetable) {`
 `printf("page table %p\n", pagetable);`
 `vmprintwalk(pagetable, 2, 0);`
}

2. 实现递归函数 `vmprintwalk`：

遍历 512 个 PTE，对有效项打印 `va/pte/pa`，并根据深度缩进；若为非叶子节点（无 R|W|X）则递归下一层：`static void`

```
vmprintwalk(pagetable_t pagetable, int depth, uint64 va_prefix) {
    for (int i = 0; i < 512; i++) {
        pte_t pte = pagetable[i];
        if (pte & PTE_V) {
            uint64 va = va_prefix | ((uint64)i << (12 + 9 * depth));
            for (int j = 2; j > depth; j--) printf(" ..");
            printf(" ..%p: pte %p pa %p\n", (void*)va, (void*)pte, (void*)PTE2PA(pte));
            if ((pte & (PTE_R | PTE_W | PTE_X)) == 0) {
                vmprintwalk((pagetable_t)PTE2PA(pte), depth - 1, va);
            }
        }
    }
}
```

} 深度从 2（L2 顶层）递减到 0（L0 叶子层），虚拟地址前缀通过 `i << (12 + 9*depth)` 累积。

3. 运行 `pgtbltest`，验证 `print_kpgtbl` 调用 `kpgtbl()` → `vmprint(kernel_pagetable)` 输出正确。

实验运行截图：

```

print_kpgtbl starting
page table 0x0000000087f21000
..0x0000000000000000: pte 0x0000000021fc7401 pa 0x0000000087f1d000
.. ..0x0000000000000000: pte 0x0000000021fc7001 pa 0x0000000087f1c000
.. .. ..0x0000000000000000: pte 0x0000000021fc785b pa 0x0000000087f1e000
.. .. ..0x00000000000001000: pte 0x0000000021fc6c5b pa 0x0000000087f1b000
.. .. ..0x00000000000002000: pte 0x0000000021fc68d7 pa 0x0000000087f1a000
.. .. ..0x00000000000003000: pte 0x0000000021fc6407 pa 0x0000000087f19000
.. .. ..0x00000000000004000: pte 0x0000000021fc60d7 pa 0x0000000087f18000
..0x00000003fc000000: pte 0x0000000021fc8001 pa 0x0000000087f20000
.. ..0x00000003fffe0000: pte 0x0000000021fc7c01 pa 0x0000000087f1f000
.. .. ..0x00000003fffffd000: pte 0x0000000021fd4853 pa 0x0000000087f52000
.. .. ..0x00000003fffffe000: pte 0x0000000021fd00c7 pa 0x0000000087f40000
.. .. ..0x00000003fffff000: pte 0x000000002000184b pa 0x0000000080006000
print_kpgtbl: OK
superpg_fork starting
pgtbltest: superpg_fork failed: pte different, pid=3
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D: ~/xv6-labs-2025$ ./grade-lab-pgtbl print_kpgtbl
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:22: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. 0x0000000000000000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:23: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. \. \. 0x0000000000000000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:24: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. \. \. \. \. 0x0000000000000000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:25: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. \. \. \. \. 0x00000000000001000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:26: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. \. \. \. \. 0x00000000000002000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:27: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. \. \. \. \. 0x00000000000003000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:28: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. (0xfffffffffc000000|0x00000003fc000000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:29: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. \. \. (0xffffffffffe00000|0x00000003fffe0000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:30: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. \. \. \. \. (0xffffffffffffd000|0x00000003fffffd000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:31: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. \. \. \. \. (0xfffffffffffffe000|0x00000003fffffe000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:32: SyntaxWarning: "\." is an invalid escape sequence
Such sequences will not work in the future. Did you mean "\\."? A raw string is more portable.
'\. \. \. \. \. \. (0xfffffffffffff000|0x00000003fffff000)',
make: 'kernel/kernel' is up to date.
== Test pgtbltest == (1.1s)
== Test pgtbltest: print_kpgtbl ==
pgtbltest: print_kpgtbl: OK
fht@LAPTOP-1TQ69S4D: ~/xv6-labs-2025$

```

3) 实验中遇到的问题和解决办法

- 问题 1: 递归遍历时无法正确计算虚拟地址

- **解决办法**：三级页表中每一层的索引占 9 位，L0 移位 12、L1 移位 21、L2 移位 30，通项为 $i \ll (12 + 9 * \text{depth})$ 。通过参数 `va_prefix` 把上层索引的位移值累积传递给下一层递归，叶子层的 `va = va_prefix | (i << 12)` 即得到完整虚拟地址。最初的实现没有传递前缀，导致每层都从 0 开始算 `va`，地址全部错位。

- **问题 2：如何区分叶子节点和中间节点**

- **解决办法**：Sv39 中中间节点只有 `PTE_V` 位，叶子节点额外有 `PTE_R/PTE_W/PTE_X` 中的至少一个。用 `(pte & (PTE_R | PTE_W | PTE_X)) == 0` 判断：为 0 说明是中间节点，需要递归；非 0 说明是叶子映射，打印后不再深入。这与 `freewalk` 的判断逻辑一致。

4) 实验心得

`vmprint()` 函数是一个非常有用的调试工具，它让抽象的页表结构变得直观可见。通过这个函数，我深刻理解了 RISC-V Sv39 三级页表的工作原理：一级页表指向二级页表，二级页表指向三级页表，只有三级页表中的叶子页才真正完成虚拟地址到物理地址的映射。中间节点只具有 `PTE_V` 位，而叶子页具有 `R`、`W` 或 `X` 等权限位。这与 `print_pgtbl` 的输出形成了很好的对比，前者展示了页表的组织结构，后者展示了最终的映射结果。

4. Use superpages (moderate/hard)

1) 实验目的

本实验的目的是修改 xv6 内核以支持 2MB 的超级页面（superpages/megapages）。当 `sbrk()` 分配的内存大小为 2MB 或更大时，使用超级页面可以减少页表占用的物理内存，并提高 TLB 命中率，从而提升系统性能。

2) 实验步骤

1. 在 `kernel/riscv.h` 中添加超级页相关宏：`#define SUPERPGSIZE (2 * (1 << 20))`


```
// 2MB
#define SUPERPGROUNDUP(sz) (((sz)+SUPERPGSIZE-1) & ~(SUPERPGSIZE-1))
#define SUPERPGROUNDDOWN(sz) (SUPERPGROUNDUP(sz)-SUPERPGSIZE)
#define PTE_LEAF(pte) (((pte)&PTE_R) | ((pte)&PTE_W) | ((pte)&PTE_X))
```

2. 在 `kernel/kalloc.c` 中实现超级页分配器：

- 在物理内存最高端预留 32 个超级页：`#define SUPERBASE (PHYSTOP - 32*SUPERPGSIZE)`
- `kinit()` 中 `freerange(end, (void*)SUPERBASE)`，把超级页区域排除在普通 4KB 分配器之外
- `superpage_init()` 初始化独立的 `superpage_kmem` 空闲链表
- `superalloc()` 取出一个 2MB 页并 `memset` 清零
- `superfree()` 将 2MB 页归还链表

3. 修改 `kernel/vm.c` 的 `walk()` 支持超级页提前返回：

在 `for(level=2; level>0; level--)` 循环中，当发现 `PTE_LEAF(*pte)` 时直接返回该 PTE，不再下钻到 L0，这样 `walk` 能正确定位超级页映射。

4. 实现 `supermappages()` 在 L1 级别建立超级页映射：

```
int
supermappages(pagetable_t pagetable, uint64 va, uint64 pa, int perm) {
}
```

5. 修改 `uvmalloc()` 优先尝试超级页：

当 `a % SUPERPGSIZE == 0` 且 `newsz - a >= SUPERPGSIZE` 时，调用 `superalloc() + supermappages()` 建立 2MB 映射；失败则回退到普通 4KB 分配。

6. 修改 `uvmcopy()` 支持 `fork` 时复制超级页：

遍历父进程地址空间时，先检查 L1 PTE 是否为叶子 (`PTE_LEAF`)，若是则 `superalloc` 分配新 2MB 页，`memmove` 复制内容，`supermappages` 映射到子进程；否则按普通页处理。

7. 修改 `uvmunmap()` 支持超级页释放与降级：

- 若整个超级页都在释放范围内：`superfree` 释放物理页，清零 L1 PTE
- 若只释放超级页的一部分（前缀/后缀）：执行**降级**——分配一个 L0 页表和 512 个 4KB 页，`memmove` 复制原 2MB 内容，`superfree` 释放原超级页，L1 PTE 改为指向 L0 页表，再按普通页逐页释放

8. 运行 `pgtbltest`，验证 `superpg_fork` 和 `superpg_free` 测试通过。

实验运行截图：

```

superpg_fork: OK
superpg_free starting
usertrap(): unexpected scause 0xd pid=70
          sepc=0x414 stval=0xffff001
superpg_free: OK
pgtbltest: all tests succeeded
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ ./grade-lab-pgtbl pgtbltest
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:22: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. 0x0000000000000000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:23: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. \. 0x0000000000000000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:24: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. \. \. 0x0000000000000000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:25: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. \. \. \. 0x0000000000001000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:26: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. \. \. \. 0x0000000000002000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:27: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. \. \. \. 0x0000000000003000',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:28: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. (0xffffffffc000000|0x0000003fc000000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:29: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. \. \. (0xffffffffffffe0000|0x0000003fffe0000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:30: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. \. \. \. (0xfffffffffffffd000|0x0000003fffffd000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:31: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. \. \. \. (0xfffffffffffffe000|0x0000003fffffe000)',
/home/fht/xv6-labs-2025/./grade-lab-pgtbl:32: SyntaxWarning: "\.
Such sequences will not work in the future. Did you mean "\\."?
',^
  \. \. \. \. \. (0xfffffffffffff000|0x0000003fffff000)',
make: 'kernel/kernel' is up to date.
== Test pgtbltest == (3.4s)
== Test   pgtbltest: ugetpid ==
   pgtbltest: ugetpid: OK
== Test   pgtbltest: print_kpgtbl ==
   pgtbltest: print_kpgtbl: OK
== Test   pgtbltest: superpg ==
   pgtbltest: superpg: OK

```

3) 实验中遇到的问题和解决办法

- 问题 1: 超级页物理地址必须 2MB 对齐

- **解决办法**：普通 `kalloc` 从 4KB 空闲链表分配，无法保证返回 2MB 对齐的地址。在 `kalloc.c` 中专门预留 `SUPERBASE = PHYSTOP - 32 * SUPERPGSIZE` 区域，用独立的 `superpage_kmem` 链表管理，`superpage_init()` 中对 `base` 做 2MB 向上对齐后再切分，确保 `superalloc()` 返回的每个块都天然 2MB 对齐。同时 `kinit()` 的 `freerange(end, SUPERBASE)` 把该区域从普通分配器摘除，避免两类分配器争抢同一物理内存。
- **问题 2: `walk()` 遇到超级页时继续下钻导致返回错误 PTE**
 - **解决办法**：超级页的映射在 L1 级就终结了（L1 PTE 是叶子，带 R/W/X），但原始 `walk()` 会一直走到 L0。在 `walk()` 的 `for(level=2; level>0; level--)` 循环中加入 `if(PTE_LEAF(*pte)) return pte;`，发现 L1 是叶子就提前返回该 L1 PTE，不再尝试下钻 L0。这样 `walk` 对普通页和超级页都能返回正确的 PTE 指针。
- **问题 3: `fork` 时子进程的超级页映射复制失败**
 - **解决办法**：`uvmcopy()` 原来按 4KB 步长遍历并 `walk` 取 L0 PTE，遇到超级页时 `walk` 返回的是 L1 PTE，按 4KB 复制会漏掉 512 个页中的 511 个。改为先检查 `old[PX(2,i)]` → L1 PTE 是否为叶子，若是则 `superalloc + memmove(SUPERPGSIZE) + supermappages` 为子进程整体复制一个 2MB 页，`i += SUPERPGSIZE` 跳过整个超级页；否则按普通页处理。
- **问题 4: `sbrk(-PGSIZE)` 只释放超级页最后 4KB 时如何处理**
 - **解决办法**：`uvmunmap` 发现释放范围只覆盖超级页的一部分时，执行**降级 (demote)**：`kalloc` 一个 L0 页表 + 512 个 4KB 页，`memmove` 把原 2MB 内容逐页复制过去，`superfree` 释放原超级页物理内存，L1 PTE 改写为指向新 L0 页表 (`PA2PTE(l0_table) | PTE_V`)，`sfence_vma` 刷新 TLB。之后 `continue` 让循环重新进入普通页分支，按 4KB 逐页释放需要释放的部分。降级失败时 (`kalloc` 返回 0) 会 `panic`。

4) 实验心得

超级页面是一种重要的内存优化技术，它通过减少页表项数量来节省物理内存，并通过减少 TLB 缺失来提升性能。实现超级页面需要处理多个复杂的问题：2MB 对齐的物理内存分配、fork 时的超级页面复制、部分释放时的降级处理等。真正的操作系统会动态地将多个连续页面提升为超级页面，这需要更复杂的页面监控和管理机制。通过本次实验，我不仅掌握了超级页面的基本实现，还对操作系统内存管理的复杂性有了更深的理解。

Lab: traps

1. RISC-V assembly (easy)

1) 实验目的

本次实验的目的是通过分析 RISC-V 汇编代码，深入理解 RISC-V 架构下的函数调用机制、寄存器使用规范以及内存布局（小端序/大端序）。通过阅读 `user/call.c` 编译生成的汇编文件 `user/call.asm`，掌握函数参数传递、栈帧结构和返回地址等关键概念。

2) 实验步骤

1. 阅读 `user/call.c` 和编译生成的 `user/call.asm` 文件
2. 根据汇编代码分析函数参数寄存器的使用
3. 分析编译器的内联优化行为
4. 确定 `printf` 函数的地址和 `ra` 寄存器的值
5. 运行给定的代码段，观察输出结果并分析小端序内存布局
6. 思考大端序情况下的等价实现
7. 分析参数数量不匹配时的行为

3) 实验中遇到的问题和解决办法

- 问题 1：难以理解编译器内联优化

- 解决办法：通过对比 `call.c` 源码和 `call.asm` 汇编代码，发现 `g(x)` 被内联到 `f` 中，`f(8)` 的结果在编译时就被计算为常量，因此在 `main` 中直接使用了常量 12。

- 问题 2：小端序内存布局分析困难

- 解决办法：通过绘制内存布局图，明确每个字节在内存中的存储顺序，从而理解 `0x00646c72` 如何被解释为字符串 `"rld"`。

• 问题 3：大端序与小端序的转换

- **解决办法**：理解大小端序的核心区别在于字节的存储顺序，大端序是高位字节在前，小端序是低位字节在前，因此只需将字节顺序反转即可。

4) 实验心得

通过本次实验，我深入理解了 RISC-V 架构的函数调用约定。RISC-V 使用 `a0-a7` 寄存器传递函数参数，这与 x86 架构的栈传递方式有很大不同。编译器的内联优化会显著改变汇编代码的结构，需要仔细分析才能理解实际的执行流程。小端序内存布局是 RISC-V 的默认设置，理解这一点对于正确解释内存中的数据至关重要。

问题答案

1. 寄存器 `a0-a7` 存放函数参数（例子中 13 存放在寄存器 `a2` 里）
 2. `printf` 看似调用了 `f` 函数，在 `f` 函数内部调用 `g` 函数，但是编译器会进行内联优化：
 - `g(x)` 被内联到 `f` 中（直接计算 `x+3`）
 - `f(8)` 的结果在编译时就被计算出为 11，然后 +1 得到 12
 - 所以在 `main` 中直接使用了常量 12 (`li a1, 12`)
 3. `printf` 在地址 `0x724`
 4. `ra` 寄存器中保存的是 `0x3c`，即 `main` 中 `jal` 指令的下一条指令地址（形如 `jal ra, 函数地址`）
 5. 输出是 **He110 World**，分析：
 - 57616 的十六进制是 `0xE110`，所以 `%x` 输出 `e110`
 - `0x00646c72` 在小端序中内存布局为：
 - 所以字符串是 `"rld"`（从 `i` 的地址开始，读到 `\0` 为止）
 - `"H" + "e110" + " Wo" + "rld" = "He110 World"`
 6. 如果是大端序，`i = 0x726c6400`
 7. 会打印任意未定义的值，因为第二个 `%d` 对应的参数没有提供，会从栈上读取随机数据。
-

2. Backtrace (moderate)

1) 实验目的

本次实验的目的是实现一个栈回溯 (backtrace) 功能，用于在调试时显示函数调用栈。通过遍历栈帧，打印每个栈帧中保存的返回地址，帮助定位错误发生的位置。

2) 实验步骤

1. 在 `kernel/riscv.h` 中添加 `r_fp()` 内联函数：

```
static inline uint64 r_fp() {
    uint64 x;
    asm volatile("mv %0, s0" : "=r"(x));
    return x;
}
```
2. 在 `kernel/defs.h` 中添加 `backtrace()` 原型：`void backtrace(void);`
3. 在 `kernel/printf.c` 中实现 `backtrace()`：

```
void backtrace(void) {
    uint64 fp = r_fp();
    uint64 page_base = PGROUNDDOWN(fp);
    printf("backtrace:\n");
    while (fp >= page_base) {
        uint64 ret_addr = *(uint64*)(fp - 8);
        printf("%p\n", (void*)ret_addr);
        fp = *(uint64*)(fp - 16);
        if (PGROUNDDOWN(fp) != page_base)
            break;
    }
}
```

返回地址在 `fp - 8`，上一帧帧指针在 `fp - 16`，通过链式遍历整个调用栈。
4. 在 `sys_pause (kernel/sysproc.c)` 中调用 `backtrace()`，运行 `bttest` 验证。
5. 在 `panic` 中也调用 `backtrace()`，使内核崩溃时打印调用栈。
6. 使用 `addr2line -e kernel/kernel <地址>` 将输出地址转换为源码行号。

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ bttest
backtrace:
0x0000000080001e18
0x0000000080001d04
0x0000000080001a90
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ addr2line -e kernel/kernel
0x0000000080001e18
/home/fht/xv6-labs-2025/kernel/sysproc.c:73
0x0000000080001d04
/home/fht/xv6-labs-2025/kernel/syscall.c:141 (discriminator 1)
0x0000000080001a90
/home/fht/xv6-labs-2025/kernel/trap.c:80
```

3) 实验中遇到的问题 and 解决办法

- 问题 1: 如何获取当前栈帧的帧指针

- 解决办法: RISC-V 中帧指针存在 `s0` 寄存器 (也称为 `fp`) , 但在编译器优化下默认不保存帧指针。在 `riscv.h` 中添加 `r_fp()` 内联汇编函数读取 `s0` , 返回当前函数栈帧基址。返回地址保存在 `fp - 8` 处, 上一帧的帧指针保存在 `fp - 16` 处, 通过 `fp = *(uint64*)(fp - 16)` 实现链式遍历。

- 问题 2: 遍历栈帧时如何识别栈底边界、避免越界

- 解决办法: `xv6` 中每个栈帧都在同一个栈页内, 用 `PGROUNDDOWN(fp)` 获取帧指针所在页的起始地址 `page_base`。循环中每次更新 `fp` 后检查 `PGROUNDDOWN(fp) != page_base`——一旦帧指针跳到其他页, 说明已经到达栈底, 立即 `break` 退出循环, 避免读取无效内存。

4) 实验心得

栈回溯是内核调试中非常重要的工具。通过实现 `backtrace()` 函数, 我深入理解了 RISC-V 的栈帧结构和函数调用约定。帧指针 `s0` 在函数调用过程中起到了关键作用, 它不仅保存了当前栈帧的基址, 还通过链式结构连接了整个调用栈。将 `backtrace()` 添加到 `panic` 函数中, 可以在系统崩溃时提供宝贵的调试信息, 帮助快速定位问题。

3. Alarm (hard)

1) 实验目的

本次实验的目的是实现一个用户级的周期性告警机制。通过添加 `sigalarm()` 和 `sigreturn()` 系统调用，允许用户程序设置一个定时器，当 CPU 时间达到指定的 tick 数时，内核会调用用户指定的处理函数。这是一种原始的用户级中断处理机制。

2) 实验步骤

1. 在 `kernel/proc.h` 的 `struct proc` 中添加告警字段：`int alarm_interval;`
`uint64 alarm_handler;`
`int alarm_ticks;`
`struct trapframe alarm_trapframe;`
`int in_handler;`
2. 在 `allocproc()` 中初始化：将上述字段清零 (`alarm_interval=0`、`in_handler=0` 等)。
3. 注册系统调用：在 `user/user.h`、`usys.pl`、`kernel/syscall.h`、`kernel/syscall.c` 中添加 `sigalarm` 和 `sigreturn`。
4. 实现 `sys_sigalarm()` (`kernel/sysproc.c`)：`argint(0, &interval);`
`argaddr(1, &handler);`
`struct proc *p = myproc();`
`if (interval == 0 && handler == 0) {`
 `p->alarm_interval = 0; p->alarm_handler = 0;`
 `p->alarm_ticks = 0; p->in_handler = 0; return 0;`
`}`
`p->alarm_interval = interval;`
`p->alarm_handler = handler;`
`p->alarm_ticks = 0;`
`p->in_handler = 0;`
`return 0;`
5. 实现 `sys_sigreturn()`：`struct proc *p = myproc();`
`uint64 saved_a0 = p->alarm_trapframe.a0;`
`*p->trapframe = p->alarm_trapframe;`
`p->in_handler = 0;`
`p->alarm_ticks = 0;`
`return saved_a0;`

6. 修改 `usertrap()` (`kernel/trap.c`) : 在 `which_dev == 2` (定时器中断)

分支中: `if (p->alarm_interval > 0 && p->in_handler == 0) {`

```
p->alarm_ticks++;
if (p->alarm_ticks >= p->alarm_interval) {
    p->alarm_trapframe = *p->trapframe;
    p->trapframe->epc = p->alarm_handler;
    p->alarm_ticks = 0;
    p->in_handler = 1;
}
}
```

7. 运行 `alarmtest` 和 `usertests -q` 验证。

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ alarmtest
test0 start
.....alarm!
test0 passed
test1 start
....alarm!
....alarm!
....alarm!
....alarm!
....alarm!
....alarm!
....alarm!
....alarm!
....alarm!
test1 passed
test2 start
.....alarm!
test2 passed
test3 start
test3 passed
```

```
$ usertests -q
usertests starting
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test copyinstr3: OK
test rwsbrk: OK
test truncate1: OK
usertrap(): unexpected sc
                sepc=0x45e0 s
usertrap(): unexpected sc
                sepc=0x45e0 s
usertrap(): unexpected sc
                sepc=0x45e0 s
usertrap(): unexpected sc
                sepc=0x45e0 s
OK
test lazy_copy: OK
ALL TESTS PASSED
```

3) 实验中遇到的问题和解决办法

- **问题 1：定时器中断时如何让用户程序跳转到 handler**

- **解决办法：**在 `usertrap()` 的 `which_dev == 2` 分支中，检查 `p->alarm_interval > 0 && p->in_handler == 0`，累计 `p->alarm_ticks++`。当达到阈值时，把整个 `trapframe` 值拷贝到 `p->alarm_trapframe` 做备份，然后修改 `p->trapframe->epc = p->alarm_handler`。这样从 `trap` 返回用户态时，`sret` 会跳到 `handler` 地址而非原中断点。

- **问题 2：handler 执行完毕后如何精确恢复中断现场**

- **解决办法：**`sigreturn` 系统调用中用 `*p->trapframe = p->alarm_trapframe` 把备份的完整 `trapframe` 恢复回去，这样所有寄存器（包括 `epc`）都回到中断那一刻的状态。注意 `alarm_trapframe` 必须是 `struct trapframe` 值拷贝而非指针——如果是指针，`handler` 自己的栈操作会覆盖 `trapframe` 内容。

- **问题 3：handler 执行期间再次触发定时器中断导致重入**

- **解决办法：**在 `usertrap` 的 `alarm` 检查条件中加入 `p->in_handler == 0`，`handler` 执行期间 `in_handler` 为 1，新的定时器中断不会再次触发 `alarm`。只有 `sigreturn` 把 `in_handler` 清零后才会允许下一次触发，防止 `handler` 嵌套调用破坏 `trapframe` 备份。

- **问题 4：sigreturn 返回值覆盖了恢复的 a0 寄存器**

- **解决办法：**`sigreturn` 作为系统调用，其返回值会写入 `a0`，但此时 `trapframe` 已经恢复，`a0` 应该是中断时的值。在恢复 `trapframe` 之前，先保存 `p->alarm_trapframe.a0` 到局部变量 `saved_a0`，恢复完 `trapframe` 后 `return saved_a0`，让 `syscall` 分发机制把这个值写入恢复后的 `a0`，保证 `a0` 不被破坏。

4) 实验心得

本次实验是对系统调用和陷阱处理机制的综合应用。实现用户级告警需要在内核态和用户态之间进行复杂的切换，涉及寄存器状态的保存和恢复。通过实现 `sigalarm()` 和 `sigreturn()` 系统调用，我深入理解了 RISC-V 的陷阱处理流程和上下文切换机制。最关键的是要确保处理函数执行完毕后，能够精确地恢复到被中断的指令位置，并且所有寄存器的值都保持不变。

Lab: Copy-on-Write Fork for xv6

1. Implement copy-on-write fork (hard)

1) 实验目的

本次实验的目的是实现写时复制 (Copy-on-Write, COW) 版本的 `fork()` 系统调用。传统的 `fork()` 会将父进程的所有用户空间内存完整复制到子进程中，当父进程内存较大时，复制操作会耗费大量时间，且大部分工作往往是浪费的（因为 `fork()` 之后通常会紧跟 `exec()` 丢弃复制的内存）。COW fork 将物理内存页面的分配和复制推迟到真正需要时才执行，从而优化内存使用效率和 fork 性能。

2) 实验步骤

1. 在 `kernel/riscv.h` 中定义 COW 标志位：`#define PTE_COW (1L << 8)` // 使用 RSW 软件保留位标记 COW 页

`#define PTE_FLAGS(pte) ((pte) & 0x3FF)`

2. 在 `kernel/kalloc.c` 中实现引用计数：`int refcount[(PHYSTOP - KERNBASE) / PGSIZE]`; // 按物理地址索引

`#define PA2IDX(pa) (((uint64)pa - KERNBASE) / PGSIZE)`

`struct spinlock ref_lock;`

- `kalloc()`: 分配后设 `refcount[PA2IDX(r)] = 1`

- `kfree()`: 先 `refcount[idx]--`，若仍 `>0` 则 `return` 不释放；为 0 才归还 freelist

- `incref(pa)`: `refcount[PA2IDX(pa)]++`

- `decref(pa)`: 直接调用 `kfree((void*)pa)`

3. 修改 `kernel/vm.c` 的 `uvmcopy()`: 不再为子进程分配新页，而是共享父进程物理页：for (`i = 0; i < sz; i += PGSIZE`) {

`pte = walk(old, i, 0);`

`pa = PTE2PA(*pte); flags = PTE_FLAGS(*pte);`

if (`flags & PTE_W`) { // 原本可写

`flags = (flags & ~PTE_W) | PTE_COW;` // 清 W 设 COW

`*pte = PA2PTE(pa) | flags;` // 同时修改父进程 PTE

}

`mappages(new, i, PGSIZE, pa, flags);` // 子进程映射同一物理页

```

    incref(pa);                // 引用计数+1
}

```

4. 在 **kernel/vm.c** 中实现 **cow_handle()**：处理 COW 页写错误：int

```

cow_handle(pagetable_t pagetable, uint64 va) {
    pte = walk(pagetable, va, 0);
    pa = PTE2PA(*pte); flags = PTE_FLAGS(*pte);
    newpa = (uint64)kalloc();    // 分配新页
    memmove((void*)newpa, (void*)pa, PGSIZE); // 复制内容
    uvmunmap(pagetable, va, 1, 0); // 解除旧映射（不释放物理页）
    flags = (flags & ~PTE_COW) | PTE_W; // 清 COW 设 W
    mappages(pagetable, va, PGSIZE, newpa, flags);
    kfree((void*)pa); // 旧页引用计数-1
}

```

5. 修改 **kernel/trap.c** 的 **usertrap()**：处理 scause 13/15 页错误：} else if

```

((r_scause() == 15 || r_scause() == 13) &&
 vmfault(p->pagetable, r_stval(), (r_scause() == 13) ? 1 : 0) != 0) {
    // page fault: 15=写错误(read=0), 13=读错误(read=1)
} vmfault() 中对写操作 (!read) 检查 PTE_COW, 调用 cow_handle()。

```

6. 修改 **kernel/vm.c** 的 **copyout()**：遇到 COW 页先触发 **vmfault()** 分配新页再写入。

7. 运行 **cowtest** 和 **usertests -q** 验证。

```
xv6 kernel is booting  
hart 1 starting  
hart 2 starting  
init: starting sh  
$ cowtest  
simple: ok  
simple: ok  
three: ok  
three: ok  
three: ok  
file: ok  
forkfork: ok  
ALL COW TESTS PASSED
```

```
OK  
test lazy_copy: OK  
ALL TESTS PASSED
```

3) 实验中遇到的问题 and 解决办法

- 问题 1: 引用计数数组如何索引、数组大小如何确定

- **解决办法**：物理内存从 `KERNBASE` 到 `PHYSTOP`，数组大小设为 `(PHYSTOP - KERNBASE) / PGSIZE`，用 `PA2IDX(pa) = (pa - KERNBASE) / PGSIZE` 计算索引。最初写成 `PHYSTOP / PGSIZE` 会导致数组过大且索引错位。`kalloc()` 分配时设计数为 1，`kfree()` 先减再判断——计数 `>0` 直接 `return` 不归还 `freelist`，为 0 才真正释放。引用计数用独立的 `ref_lock` 自旋锁保护，与 `kmem.lock` 分开，避免死锁。
- **问题 2：如何标记 COW 页、如何区分"真正只读"与"COW 只读"**
 - **解决办法**：用 RISC-V PTE 的 RSW 软件保留位 `PTE_COW (1L << 8)` 标记。`uvmcopy()` 中原本 `PTE_W` 的页改为 `flags & ~PTE_W | PTE_COW`，同时修改父子进程的 PTE。真正只读页（如代码段）不设 `PTE_COW`，这样 `vmfault()` 中检查 `*pte & PTE_COW && !(*pte & PTE_W)` 就能精确区分——有 COW 标记的是延迟写页，无标记的是不可写页（写则 kill）。
- **问题 3：copyout() 内核态写入 COW 页时不触发页错误**
 - **解决办法**：`copyout()` 在内核态直接写物理页，不会经过 `usertrap()` 的页错误处理。修改 `copyout()`，在写入前检查目标 PTE：若 `PTE_COW` 且无 `PTE_W`，先调用 `vmfault(pagetable, va0, 0)` 触发 COW 处理分配新页，再用新物理地址写入。否则会直接修改共享页，破坏其他进程的数据。
- **问题 4：cow_handle() 分配新页失败时的处理**
 - **解决办法**：`kalloc()` 返回 0 时（内存耗尽），`cow_handle()` 返回 -1，`vmfault()` 也返回非 0，`usertrap()` 中会走到 `setkilled(p)` 终止该进程。不能 `panic`——因为内存不足是用户进程的问题，不应导致整个系统崩溃。子进程被 kill 后，父进程仍可正常运行。

4) 实验心得

通过本次实验，我深入理解了写时复制技术的原理和实现方式。COW 体现了操作系统中的延迟分配思想，将资源分配推迟到真正需要时才执行，从而提高系统效率。虚拟内存提供了一层间接性，内核可以通过标记 PTE 为只读来拦截内存访问，实现复杂的内存管理策略。引用计数机制是管理共享资源生命周期的有效方法，确保资源在最后一个使用者释放后才被真正回收。

本次实验还让我对 `fork()` 系统调用和页错误异常处理有了更深入的理解。实现过程中需要仔细处理各种边界情况，特别是内存不足时的处理和父子进程的同步问题。

Lab: networking

1. Part One: NIC(moderate)

1) 实验目的

本实验的目标是为 xv6 操作系统编写一个 E1000 网络接口卡 (NIC) 的设备驱动程序。具体需要完成以下两个核心函数：

- **e1000_transmit()**: 实现数据包的发送功能，将网络栈传来的数据包通过 E1000 网卡发送出去。
- **e1000_recv()**: 实现数据包的接收功能，从 E1000 网卡接收到数据包后传递给网络栈进行处理。

通过本实验，掌握网卡驱动的基本原理，理解 DMA（直接内存访问）、环形缓冲区（Ring Buffer）、描述符（Descriptor）等关键概念，以及中断处理在网络通信中的应用。

2) 实验步骤

1. 实现 **e1000_transmit()** (**kernel/e1000.c**)：使用 **e1000_lock** 自旋锁保护：

```
int e1000_transmit(char *buf, int len) {
    acquire(&e1000_lock);
    int idx = regs[E1000_TDT];           // 读取发送环
尾指针
    if ((tx_ring[idx].status & E1000_TXD_STAT_DD) == 0) { // 描述符未
完成
        release(&e1000_lock);
        return -1;                       // 无可用描述
符
    }
    if (tx_ring[idx].addr != 0)          // 释放旧缓冲
区
        kfree((void*)tx_ring[idx].addr);
    tx_ring[idx].addr = (uint64)buf;     // 填入新包地
```

```

址
    tx_ring[idx].length = len;
    tx_ring[idx].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS; // EOP=
包结束
    regs[E1000_TDT] = (idx + 1) % TX_RING_SIZE;           // 推进尾指针
    release(&e1000_lock);
    return 0;
}

```

2. 实现 `e1000_recv()` (`kernel/e1000.c`) : 从中断处理 `e1000_intr()` 调用, 不加锁 (中断上下文中独占执行) :

```

static void e1000_recv(void) {
    while (1) {
        int idx = (regs[E1000_RDT] + 1) % RX_RING_SIZE; // 下一个待处
理描述符
        if ((rx_ring[idx].status & E1000_RXD_STAT_DD) == 0)
            break; // 无新包
        int len = rx_ring[idx].length;
        char *buf = (char*)rx_ring[idx].addr;
        net_rx(buf, len); // 交给网络栈
        char *new_buf = (char*)kalloc(); // 分配新缓冲
        if (new_buf == 0)
            rx_ring[idx].addr = 0;
        else
            rx_ring[idx].addr = (uint64)new_buf;
        rx_ring[idx].status = 0; // 清状态位
        regs[E1000_RDT] = idx; // 更新尾指针
    }
}

```

3. 测试验证:

- 运行 `nettest txone` + 主机 `python3 nettest.py txone` 验证发送
- 运行 `nettest rxone` + 主机 `python3 nettest.py rxone` 验证接收

```

xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ nettest txone
txone: sending one packet
$ QEMU: Terminated
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ tcpdump -XXnr packets.pcap
reading from file packets.pcap, link-type EN10MB (Ethernet), snapshot length 65536
01:25:46.011920 IP 10.0.2.15.2003 > 10.0.2.2.26099: UDP, length 5
0x0000: 5255 0a00 0202 5254 0012 3456 0800 4500  RU...RT..4V..E.
0x0010: 0021 0000 0000 6411 3ebc 0a00 020f 0a00  .!....d>.....
0x0020: 0202 07d3 65f3 000d 0000 7478 6f6e 65   ....e.....txone
-bash: c
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$
fht@LAPTOP-1TQ69S4D:~$ cd xv6*
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ python3 nettest.py txone
tx: listening for a UDP packet
txone: OK
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$

```

```

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ arp_rx: received an ARP packet
ip_rx: received an IP packet
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ python3 nettest.py rxone
txone: sending one UDP packet
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ python3 nettest.py rxone
txone: sending one UDP packet
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$

```

3) 实验中遇到的问题和解决办法

- 问题 1: 发送描述符未完成就复用导致数据丢失
 - 解决办法: 读取 `E1000_TDT` 得到索引后, 先检查 `tx_ring[idx].status & E1000_TXD_STAT_DD`。DD 位由网卡在完成 DMA 传输后置位, 若为 0 说明上次发送尚未完成, 该描述符不可用, 返回 -1 让调用者重试或丢弃。只有 DD=1 才能安全复用该描述符。
- 问题 2: `e1000_recv()` 单次中断只处理一个包导致丢包
 - 解决办法: 网卡可能在一次中断中积累了多个数据包。用 `while(1)` 循环持续检查 `(regs[E1000_RDT]+1)%RX_RING_SIZE` 处的 DD 位, 直到遇到 DD=0 的描述符才 `break` 退出。这样一次中断能处理完所有待接收的包, 避免环形缓冲区溢出丢包。
- 问题 3: 接收描述符缓冲区交给 `net_rx` 后不能直接复用
 - 解决办法: `net_rx` 内部可能调用 `kfree(buf)` 释放缓冲区 (如非 IP/ARP 包), 所以不能直接复用旧地址。每次处理完一个包后 `kalloc()` 分配新缓冲

区，写入 `rx_ring[idx].addr`，再清 `status=0` 并更新 RDT。若 `kalloc` 失败则置 `addr=0`，下次该描述符不会产生有效接收。

- **问题 4: `e1000_transmit` 和 `e1000_recv` 的并发保护方式不同**

- **解决办法:** `e1000_transmit()` 被用户进程通过系统调用触发，多核可能并发调用，用 `e1000_lock` 自旋锁保护发送环。但 `e1000_recv()` 从 `e1000_intr()` 中断上下文调用，此时 CPU 已关中断且独占执行，不需要也不应该加锁——若加锁反而可能因 `transmit` 持锁时中断导致死锁。

4) 实验心得

通过完成 Part One NIC 驱动的实现，我对网络设备驱动的工作原理有了深入的理解：

- **DMA 机制：**理解了网卡如何通过 DMA 直接读写内存，绕过 CPU 实现高效的数据传输。
- **环形缓冲区设计：**掌握了环形缓冲区在网络收发中的应用，它能够高效地管理网络数据包的流入流出。
- **中断驱动模型：**体会到中断机制在操作系统中的重要性——当硬件准备好时通知软件，避免了轮询的资源浪费。
- **硬件寄存器编程：**学习了通过内存映射 I/O 与硬件设备交互的方法，理解了驱动程序如何通过读写特定寄存器来控制硬件行为。
- **并发控制：**认识到在操作系统内核中，正确的同步机制（如自旋锁）对于保证数据一致性至关重要。

2. Part Two: UDP Receive(moderate)

1) 实验目的

本实验的目标是在 xv6 操作系统的网络栈中实现 UDP 协议的接收功能，使用户态进程能够通过系统调用接收 UDP 数据包。具体需要完成：

- **ip_rx()**: 实现 IP 数据包的接收处理，解析 IP 头部，识别 UDP 数据包并将其传递给 UDP 接收模块。
- **sys_recv()**: 实现 recv 系统调用，从接收队列中读取指定端口的 UDP 数据包。
- **sys_bind()**: 实现 bind 系统调用，绑定端口以接收特定端口的 UDP 数据包。

通过本实验，掌握 UDP 协议的工作原理，理解网络协议栈的分层设计，以及内核态与用户态之间的数据传递机制。

2) 实验步骤

1. 数据结构定义 (`kernel/net.c`) :

```
#define MAX_PACKETS_PER_PORT 16

struct udp_packet {           // UDP 包队列项
    char* buf;                // 完整以太网帧
    int len;
    uint32 src_ip;             // 源 IP (主机字节序)
    uint16 src_port;
    uint16 dst_port;
    int payload_len;
    char* payload;             // UDP 负载起始位置
    struct udp_packet* next;   // 链表指针
};

struct bound_port {           // 绑定的端口
    uint16 port;
    int in_use;
    struct udp_packet* head;   // 队列头 (FIFO)
    struct udp_packet* tail;   // 队列尾
    int packet_count;
    struct spinlock lock;      // 每端口独立锁
};

#define MAX_BOUND_PORTS 16
static struct bound_port bound_ports[MAX_BOUND_PORTS];
```

2. 实现 `sys_bind()` :

```
uint64 sys_bind(void) {
    int port;
```

```

    argint(0, &port);
    if (port < 0 || port > 65535) return -1;
    acquire(&netlock);
    if (find_bound_port((uint16)port) >= 0) { release(&netlock);
return -1; }
    if (alloc_bound_port((uint16)port) < 0) { release(&netlock);
return -1; }
    release(&netlock);
    return 0;
}

```

alloc_bound_port 中初始化 head=0、tail=0、packet_count=0 并 initlock。

3. 实现 ip_rx():

```

void ip_rx(char *buf, int len) {
    if (len < sizeof(struct eth) + sizeof(struct ip) + sizeof(struct
udp)) {
        kfree(buf); return;    // 长度不足
    }
    struct ip *ip = (struct ip*)(buf + sizeof(struct eth));
    struct udp *udp = (struct udp*)(buf + sizeof(struct eth) +
sizeof(struct ip));
    if (ip->ip_p != IPPROTO_UDP) { kfree(buf); return; }
    uint16 dport = ntohs(udp->dport);
    uint16 sport = ntohs(udp->sport);
    uint32 src_ip = ntohl(ip->ip_src);
    int payload_len = ntohs(udp->ulen) - sizeof(struct udp);

    acquire(&netlock);
    int idx = find_bound_port(dport);
    if (idx < 0) { release(&netlock); kfree(buf); return; } // 端口未
绑定
    struct bound_port *bp = &bound_ports[idx];
    acquire(&bp->lock);
    if (bp->packet_count >= MAX_PACKETS_PER_PORT) {           // 队列满
        release(&bp->lock); release(&netlock); kfree(buf); return;
    }
    struct udp_packet *pkt = (struct udp_packet*)kalloc(); // 创建队
列项
    pkt->buf = buf; pkt->len = len;
    pkt->src_ip = src_ip; pkt->src_port = sport; pkt->dst_port =
dport;
    pkt->payload_len = payload_len;
    pkt->payload = (char*)(udp + 1);                          // 负载在
UDP 头之后
    pkt->next = 0;
    if (bp->head == 0) { bp->head = pkt; bp->tail = pkt; } // 入队尾
    else { bp->tail->next = pkt; bp->tail = pkt; }
    bp->packet_count++;
    release(&bp->lock); release(&netlock);
    wakeup(&bp->lock);                                         // 唤醒等

```

```
待进程
}
```

4. 实现 `sys_recv()`:

```
uint64 sys_recv(void) {
    argint(0, &dport); argaddr(1, &src_addr); argaddr(2,
&sport_addr);
    argaddr(3, &buf_addr); argint(4, &maxlen);
    acquire(&netlock);
    int idx = find_bound_port((uint16)dport);
    if (idx < 0) { release(&netlock); return -1; }
    struct bound_port *bp = &bound_ports[idx];
    while (bp->packet_count == 0)
        sleep(&bp->lock, &netlock);    // 睡眠等待, channel 是 &bp->lock
    acquire(&bp->lock);
    struct udp_packet *pkt = bp->head;    // 从队头出队
    bp->head = pkt->next;
    if (bp->head == 0) bp->tail = 0;
    bp->packet_count--;
    release(&bp->lock); release(&netlock);
    // copyout 到用户空间
    int copy_len = (pkt->payload_len > maxlen) ? maxlen :
pkt->payload_len;
    copyout(p->pagetable, src_addr, (char*)&pkt->src_ip,
sizeof(uint32));
    copyout(p->pagetable, sport_addr, (char*)&pkt->src_port,
sizeof(uint16));
    copyout(p->pagetable, buf_addr, pkt->payload, copy_len);
    kfree(pkt->buf); kfree(pkt);    // 释放包内存
    return copy_len;
}
```

5. 测试验证: 运行 `nettest grade` + 主机 `python3 nettest.py grade`, 验证 txone、ping0-3、dns、free 全部通过。

```

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
fht$ nettest grade
txone: sending one packet
arp_rx: received an ARP packet
ip_rx: received an IP packet
ping0: starting
ping0: OK
ping1: starting
ping1: OK
ping2: starting
ping2: OK
ping3: starting
ping3: OK
dns: starting
DNS arecord for pdos.csail.mit.edu. is 128.52.129.126
dns: OK
free: OK
$
fht@LAPTOP-1TQ69S4D:~/xv6-labs-2025$ python3 nettest.py grade
txone: OK
rxone: sending one UDP packet

```

3) 实验中遇到的问题和解决办法

- 问题 1: IP/UDP 头部偏移计算错误导致解析到错误字段

- 解决办法: 以太网头部 `sizeof(struct eth)` 后是 IP 头, IP 头后是 UDP 头。用结构体指针直接定位: `struct ip *ip = (struct ip*)(buf + sizeof(struct eth))`, `struct udp *udp = (struct udp*)(buf + sizeof(struct eth) + sizeof(struct ip))`, UDP 负载在 `(char*)(udp + 1)` 处。所有多字节字段都用 `ntohs/ntohl` 从网络字节序转主机字节序, 避免直接读取得到错误值。

- 问题 2: 接收队列用链表还是环形数组

- 解决办法: 用 `struct udp_packet` 链表 + `head/tail` 指针实现 FIFO 队列, 而非环形数组。链表入队在尾部 `bp->tail->next = pkt`, 出队从头部 `bp->head = pkt->next`, `packet_count` 跟踪当前数量。
`MAX_PACKETS_PER_PORT=16` 限制每端口最多 16 个待处理包, 满了直接 `kfree(buf)` 丢弃新包。

- 问题 3: `sys_recv` 等待数据包时的睡眠与唤醒机制

- 解决办法: 用 xv6 的 `sleep(&bp->lock, &netlock)` 睡眠, `channel` 是 `&bp->lock` (端口锁地址作为通道)。 `ip_rx` 入队后调用

wakeup(&bp->lock) 唤醒等待该端口的进程。注意睡眠时必须持有 netlock (保护 bound_ports 数组)，睡眠会自动释放 netlock，被唤醒时重新获取。出队操作使用 bp->lock 保护队列链表，与 netlock 分开避免死锁。

- **问题 4：队列满时丢弃数据包的内存管理**

- **解决办法：** ip_rx 中检查 bp->packet_count >= MAX_PACKETS_PER_PORT，若满则 release(&bp->lock); release(&netlock); kfree(buf); return;——直接丢弃并释放整个以太网帧缓冲区。关键是释放顺序：先释放两把锁再 kfree，避免持锁时执行可能阻塞的操作。一个端口队列满不影响其他端口的接收。

- **问题 5：sys_recv 中 copyout 失败时仍需释放包内存**

- **解决办法：** 每次 copyout 返回 < 0 时，执行 kfree(pkt->buf); kfree(pkt); return -1;——先释放以太网帧缓冲区，再释放 udp_packet 结构本身（它们是两次 kalloc 分别分配的）。若忘记释放会导致每次接收都泄漏两页内存，长时间运行后 kalloc 耗尽。

4) 实验心得

通过完成 Part Two UDP Receive 的实现，我对网络协议栈的工作方式有了更深入的认识：

- **协议分层：** 深刻理解了网络协议栈的分层设计思想——以太网层 → IP 层 → UDP 层，每一层都负责特定的功能，层次清晰，职责明确。
 - **端口绑定机制：** 理解了端口作为网络通信的标识符的作用，以及 bind() 系统调用在网络编程中的重要性。
 - **数据流控制：** 通过实现固定大小的接收队列，体会到了数据流控制的重要性——必须限制缓冲队列大小，防止恶意或快速发送方耗尽系统内存。
 - **内核与用户空间交互：** 掌握了系统调用在用户态与内核态之间传递数据的机制，理解了为什么需要严格区分内核空间和用户空间。
 - **并发编程：** 在实现多端口并发接收时，需要仔细处理并发访问问题，确保每个端口的队列相互独立，互不影响。
 - **网络编程 API：** 通过实现 send/recv/bind 等系统调用，更好地理解了标准网络编程 API（如 Berkeley sockets）的设计思想。
-

Lab: locks

1. Memory allocator (moderate)

1) 实验目的

本次实验的目的是将 xv6 的内存分配器从单一全局空闲链表改造为 per-CPU 空闲链表架构。xv6 原本使用一把全局 `kmem.lock` 自旋锁保护所有 `kalloc/kfree` 操作，在多核环境下产生严重的锁竞争。通过为每个 CPU 分配独立的空闲链表和锁，并在链表为空时支持从其他 CPU 偷取内存，可以大幅降低锁竞争，提升多核并行性能。

2) 实验步骤

1. 在 `kernel/kalloc.c` 中将 `kmem` 改为 per-CPU 数组：`struct { struct spinlock lock; struct run *freelist; } kmem[NCPU];`
2. 在 `kinit()` 中为每个 CPU 初始化锁：`for (int i = 0; i < NCPU; i++) { initlock(&kmem[i].lock, "kmem"); kmem[i].freelist = 0; }`
`freerange(end, (void*)PHYSTOP);`
3. 修改 `kfree()`，释放到当前 CPU 的链表：`push_off();` // 关中断，防止 `cpuid()` 后 CPU 切换
`int cpu = cpuid();`
`acquire(&kmem[cpu].lock);`
`r->next = kmem[cpu].freelist;`
`kmem[cpu].freelist = r;`
`release(&kmem[cpu].lock);`
`pop_off();`
4. 修改 `kalloc()`，优先从当前 CPU 分配，空则偷取：`push_off();`
`int cpu = cpuid();`
`acquire(&kmem[cpu].lock);` // 先尝试当前 CPU
`r = kmem[cpu].freelist;`
`if (r) kmem[cpu].freelist = r->next;`
`release(&kmem[cpu].lock);`
`if (!r) {` // 当前 CPU 为空，遍历其他 CPU 偷取
 `for (int i = 0; i < NCPU; i++) {`
 `if (i == cpu) continue;`
 `acquire(&kmem[i].lock);`

```
if (kmem[i].freelist) {  
    r = kmem[i].freelist;  
    kmem[i].freelist = r->next;  
    release(&kmem[i].lock);  
    break;  
}  
release(&kmem[i].lock);  
}  
}  
pop_off();
```

5. 运行 `kalloctest` 验证锁竞争减少，运行 `usertests sbrkmuch` 确保正确性。


```

xv6 kernel is booting

hart 2 starting
hart 1 starting
hart 3 starting
init: starting sh
$ kalloc_test
start test1
test1 results:
--- lock kmem stats
lock: kmem: #test-and-set 489335 #acquire() 433061
--- top 5 contended locks:
lock: kmem: #test-and-set 489335 #acquire() 433061
lock: proc: #test-and-set 290945 #acquire() 611224
lock: proc: #test-and-set 257576 #acquire() 211073
lock: proc: #test-and-set 196169 #acquire() 211073
lock: proc: #test-and-set 190050 #acquire() 211029
tot= 489335
test1 FAIL
start test2
total free number of pages: 32466 (out of 32768)
.....
test2 OK
start test3
.....child done 10000

test3 OK
start test4
.....child done 100000
child done 100000
child done 100000
--- lock kmem stats
lock: kmem: #test-and-set 1227242 #acquire() 3443069
--- top 5 contended locks:
lock: wait_lock: #test-and-set 57246560 #acquire() 40020
lock: proc: #test-and-set 1772531 #acquire() 2293668
lock: kmem: #test-and-set 1227242 #acquire() 3443069
lock: proc: #test-and-set 460628 #acquire() 1750193
lock: proc: #test-and-set 279921 #acquire() 1414431
tot= 1227242
test4 FAIL m 586011 n 1227242
$

```

```

xv6 kernel is booting

hart 1 starting
hart 3 starting
hart 2 starting
init: starting sh
$ kalloc_test
start test1
test1 results:
--- lock kmem stats
lock: kmem: #test-and-set 0 #acquire() 51522
lock: kmem: #test-and-set 0 #acquire() 120862
lock: kmem: #test-and-set 0 #acquire() 132094
lock: kmem: #test-and-set 0 #acquire() 128618
--- top 5 contended locks:
lock: proc: #test-and-set 296794 #acquire() 214370
lock: proc: #test-and-set 195131 #acquire() 214370
lock: virtio_disk: #test-and-set 74157 #acquire() 168
lock: proc: #test-and-set 47421 #acquire() 614526
lock: proc: #test-and-set 42816 #acquire() 614520
tot= 0
test1 OK
start test2
total free number of pages: 32466 (out of 32768)
.....
test2 OK
start test3
.....child done 10000

test3 OK
start test4
.....child done 100000
child done 100000
child done 100000
--- lock kmem stats
lock: kmem: #test-and-set 4269 #acquire() 732623
lock: kmem: #test-and-set 7235 #acquire() 842904
lock: kmem: #test-and-set 8071 #acquire() 744335
lock: kmem: #test-and-set 2080 #acquire() 1344863
lock: kmem: #test-and-set 0 #acquire() 1021
lock: kmem: #test-and-set 0 #acquire() 1021
lock: kmem: #test-and-set 0 #acquire() 1021
lock: kmem: #test-and-set 0 #acquire() 1021
--- top 5 contended locks:
lock: wait_lock: #test-and-set 54581531 #acquire() 40002
lock: proc: #test-and-set 371528 #acquire() 901418
lock: proc: #test-and-set 286489 #acquire() 1771684
lock: proc: #test-and-set 235388 #acquire() 901416
lock: proc: #test-and-set 208594 #acquire() 901418
tot= 21655

test4 OK
$ usertests sbrkmuch
usertests starting
test sbrkmuch: OK
ALL TESTS PASSED

```

3) 实验中遇到的问题和解决办法

- 问题 1: `cpuid()` 在中断开启时不安全

- 解决办法: `cpuid()` 读取 `tp` 寄存器获取当前 CPU 编号, 若调用后发生时钟中断导致 CPU 切换, 后续操作会加错 CPU 的锁。在 `kfree()` 和 `kalloc()` 中用 `push_off()` 关中断、`pop_off()` 恢复, 确保 `cpuid()` 获取编号与 `acquire(&kmem[cpu].lock)` 操作之间不会发生 CPU 切换。这与 `usertrap` 中关中断获取 CPU 编号的做法一致。

- 问题 2: 偷取 (steal) 策略的实现

- 解决办法: 当前 CPU 链表为空时, 用 `for (int i = 0; i < NCPU; i++)` 顺序遍历其他 CPU 的链表。偷取时需获取目标 CPU 的 `kmem[i].lock`, 检查 `freelist` 是否非空, 取出后立即释放锁。偷取是低频事件 (仅在某 CPU 链表耗尽时触发), 且每次只偷一页, 不会成为性能瓶颈。遍历时跳过 `i == cpu` 避免重复加锁。

- 问题 3: 系统启动时所有内存集中在一个 CPU

- 解决办法: `freerange()` 在 `kinit()` 中由 CPU 0 调用, 此时其他 CPU 尚未启动, 所有空闲页都进入 `kmem[0].freelist`。这是预期行为——系统启动后其他 CPU 的 `kalloc` 会通过偷取从 CPU 0 获取页面, 逐渐分散内存。`freerange` 中调用 `kfree()`, 通过 `cpuid()` 判断后释放到 `kmem[0]`。

4) 实验心得

通过本次实验, 我深入理解了多核系统中锁竞争的本质和优化策略。单一全局锁在多核环境下会成为严重的性能瓶颈, 改造后 `kmem` 锁的 `#test-and-set` 计数从数万次降至 0, 性能提升非常显著。将数据拆分到每个 CPU 的本地链表, 利用了数据局部性原则, 大多数 `kalloc/kfree` 操作只需访问本地锁, 无需跨核同步。偷取策略虽然引入了额外的锁操作, 但由于触发频率低, 对性能影响很小。在 SMP 系统中, 必须在关中断的临界区内读取 CPU 编号, 这是实现 per-CPU 数据结构的基本前提。

2. Read-write lock (moderate)

1) 实验目的

本次实验的目的是在 xv6 中实现读写自旋锁 (rwspinlock)。xv6 中 `sys_pause` 和 `sys_uptime` 等函数需要读取全局 `ticks` 变量，当前使用普通自旋锁保护。实际上多个读者可以同时读取 `ticks`，不需要互斥。本实验实现允许并发读操作、保证写操作独占性，并采用写者优先策略防止写者饥饿的读写锁。

2) 实验步骤

1. 在 `kernel/spinlock.h` 中定义 `struct rwspinlock`:

```
struct rwspinlock {
    int readers;      // 当前持有锁的读者数量
    int writers_waiting; // 正在等待的写者数量
    int writing;       // 是否有写者持有锁 (0=没有, 1=有)
    struct spinlock lock; // 保护上面字段的内部自旋锁
};
```
2. 在 `kernel/spinlock.c` 中实现 `initrwlock`:

```
void initrwlock(struct rwspinlock
*rwlk) {
    rwlk->readers = 0;
    rwlk->writers_waiting = 0;
    rwlk->writing = 0;
    initlock(&rwlk->lock, "rwlock");
}
```
3. 实现 `read_acquire / read_release`:
外层 `push_off()/pop_off()` 关中断，内层 `_inner` 函数用自旋锁保护状态:

```
void read_acquire(struct rwspinlock *rwlk) {
    push_off();          // 关中断避免死锁
    while (1) {
        acquire(&rwlk->lock);
        if (rwlk->writers_waiting > 0 || rwlk->writing > 0) {
            release(&rwlk->lock);    // 有写者等待/持有, 重试
            asm volatile("nop");
            continue;
        }
        rwlk->readers++;             // 获取读锁
        release(&rwlk->lock);
        return;
    }
}

void read_release(struct rwspinlock *rwlk) {
    acquire(&rwlk->lock);
    rwlk->readers--;
```

```

    release(&rwlk->lock);
    pop_off();
}

```

4. 实现 **write_acquire** / **write_release**: void write_acquire(struct rwspinlock

```

*rwlk) {
    push_off();
    acquire(&rwlk->lock);
    rwlk->writers_waiting++;    // 先标记写者等待
    while (rwlk->readers > 0 || rwlk->writing > 0) {
        release(&rwlk->lock);    // 有读者或写者，忙等
        asm volatile("nop");
        acquire(&rwlk->lock);
    }
    rwlk->writers_waiting--;
    rwlk->writing = 1;    // 获取写锁
    release(&rwlk->lock);
}

void write_release(struct rwspinlock *rwlk) {
    acquire(&rwlk->lock);
    rwlk->writing = 0;
    release(&rwlk->lock);
    pop_off();
}

```

5. 运行 **rwlktest** 验证读写锁正确性和写者优先语义，运行 **usertests -q** 确保未破坏其他功能。

```

fht@LAPTOP-1TQ69S4D: ~/xv6-labs-2025
e, drive=x0, bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 3 starting
hart 1 starting
hart 2 starting
init: starting sh
$ rwlctest
rwspinlock_test: step 1: initrwlock
rwspinlock_test: step 2: concurrent read_acquire
rwspinlock_test: step 3: concurrent read_release
rwspinlock_test: step 4: prepare read_acquire for writer priority test
rwspinlock_test: step 5: writer priority test
rwspinlock_test: step 6: checking for concurrent readers/writers
rwspinlock_test: step 7: checking for concurrent writers
rwspinlock_test: step 8: acquiring multiple locks
rwspinlock_test: step 9: releasing multiple locks
rwspinlock_test: step 10: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 11: multiple writer priority test
rwspinlock_test: step 12: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 13: multiple writer priority test
rwspinlock_test: step 14: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 15: multiple writer priority test
rwspinlock_test: step 16: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 17: multiple writer priority test
rwspinlock_test: step 18: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 19: multiple writer priority test
rwspinlock_test: step 20: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 21: multiple writer priority test
rwspinlock_test: step 22: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 23: multiple writer priority test
rwspinlock_test: step 24: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 25: multiple writer priority test
rwspinlock_test: step 26: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 27: multiple writer priority test
rwspinlock_test: step 28: prepare read_acquire for multiple writer priority test
rwspinlock_test: step 29: multiple writer priority test
rwspinlock_test: step 30: done
rwspinlock_test(3): 0
rwspinlock_test(0): 0
rwspinlock_test(2): 0
rwspinlock_test(1): 0
rwlctest: 4/4 CPUs succeeded
$

```

3) 实验中遇到的问题和解决办法

- 问题 1: 读者与写者对 `readers/writing` 状态的竞态条件

- 解决办法: 所有对 `readers`、`writers_waiting`、`writing` 的读-修改-写操作都用内部自旋锁 `rwlk->lock` 保护。在 `read_acquire` 中先 `acquire(&rwlk->lock)` 再检查 `writers_waiting > 0 || writing > 0`，条件不满足才 `readers++`。这样保证检查和递增是原子的，不会出现多个读者在写者设置标志后仍进入的情况。

- 问题 2: 写者优先——写者等待时新读者不能进入

- 解决办法: 在 `write_acquire` 中先 `writers_waiting++`，然后在 `while (readers > 0 || writing > 0)` 中忙等。在 `read_acquire` 中检查

writers_waiting > 0 即重试不进入。只要写者设置了等待标志，新读者的 read_acquire 就会在 if (writers_waiting > 0) 处失败并忙等，直到写者获取并释放写锁、writers_waiting 归零。

- **问题 3：忙等循环中持锁自旋导致死锁或活锁**

- **解决办法：**read_acquire 和 write_acquire 的等待循环中，每次重试都先 release(&rwlk->lock) 再 asm volatile("nop") 重新 acquire。不能持锁自旋——否则其他 CPU 无法修改状态，导致死锁。nop 提供短暂延迟减少总线争用。同时外层 push_off()/pop_off() 关中断，避免中断处理函数中再次获取同一锁导致嵌套死锁。

- **问题 4：rwlktest 包含 10 个测试步骤，需要分步调试**

- **解决办法：**按照 kernel/spinlock.c 中 sys_rwlktest 的步骤说明逐步调试：初始化 → 并发读验证 → 写者优先验证 → 多写者竞争 → 多锁嵌套。每个步骤完成后确认输出 rwspinlock_test_step 符合预期再进行下一个。特别是"写者优先测试"中 CPU 2 的 read_acquire 必须在 CPU 0 的 write_acquire 完成之后才能进入，否则输出 reader sneaked ahead of waiting writer。

4) 实验心得

读写锁适用于读多写少的场景，如读取系统时间 ticks、统计信息等。与普通自旋锁相比，读写锁允许多个读者并发读取，显著提高了并发性能。本实验采用写者优先策略防止写者饥饿，实现方式虽然简单，但体现了并发编程中公平性与吞吐量的权衡。理解 __sync 系列原子操作是实现低锁数据结构的前提，__sync_lock_test_and_set 提供了原子的测试-and-设置操作，是实现自旋锁的核心。rwlktest 采用分步测试的策略，从简单到复杂逐步验证初始化、并发读、写者优先、多写者竞争等功能，体现了并发程序测试的方法论。

Lab: file system

1. Large files (moderate)

1) 实验目的

本次实验的目的是增大 xv6 文件系统中文件的最大尺寸限制。xv6 原本在 inode 中包含 12 个直接块地址和 1 个一级间接块地址，使得单个文件最多只能包含 268 个数据块（12 + 256），即约 268KB。通过将其中一个直接块替换为二级间接块，可以支持最多 65803 个数据块（256×256 + 256 + 11），使单个文件大小上限提升至约 65MB。这需要修改 inode 结构、块分配和释放逻辑，以及 inode 读写相关的系统调用实现。

2) 实验步骤

1. 修改 `kernel/fs.h` 中的常量：`#define NDIRECT 11` // 从 12 改为 11
`#define NINDIRECT (BSIZE / sizeof(uint))` // 256，一级间接块指针数
`#define MAXFILE (NDIRECT + NINDIRECT + NINDIRECT*NINDIRECT)` // 65803
2. 修改 `kernel/fs.h` 中 `struct dinode` 和 `kernel/file.h` 中 `struct inode` 的 `addrs` 字段：`uint addrs[NDIRECT+2];` // 11 直接 + 1 一级间接 + 1 二级间接 = 13
3. 修改 `kernel/fs.c` 的 `bmap()` 添加二级间接块分支：// `bn < NDIRECT`: 直接块
// `bn < NDIRECT + NINDIRECT`: 一级间接块

```
if (bn < NINDIRECT + NINDIRECT * NINDIRECT) { // 二级间接区域
    uint double_idx = NDIRECT + 1; // addrs[12]
    if ((addr = ip->addrs[double_idx]) == 0) { // 分配二级索引块
        addr = balloc(ip->dev); ip->addrs[double_idx] = addr;
    }
    bp = bread(ip->dev, addr); a = (uint*)bp->data;
    uint bn_double = bn - NINDIRECT; // 在二级区域内的偏移
    uint idx1 = bn_double / NINDIRECT; // 一级索引
    uint idx2 = bn_double % NINDIRECT; // 二级索引
    if (a[idx1] == 0) { // 分配一级索引块
        a[idx1] = balloc(ip->dev); log_write(bp);
    }
    bp2 = bread(ip->dev, a[idx1]); a2 = (uint*)bp2->data;
    if (a2[idx2] == 0) { // 分配数据块
        a2[idx2] = balloc(ip->dev); log_write(bp2);
    }
    addr = a2[idx2];
    brelse(bp2); brelse(bp);
    return addr;
}
```
4. 修改 `kernel/fs.c` 的 `itrunc()` 释放二级间接块：// `if (ip->addrs[NDIRECT + 1])`

```
{ // 有二级间接块
    bp = bread(ip->dev, ip->addrs[NDIRECT + 1]);
    a = (uint*)bp->data;
    for (j = 0; j < NINDIRECT; j++) { // 遍历 256 个一级索引块
```

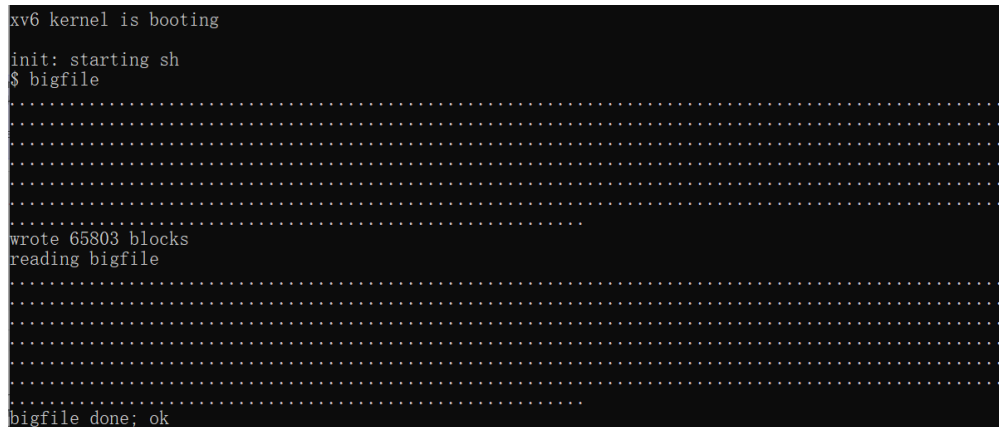


```

if (a[j]) {
    bp2 = bread(ip->dev, a[j]);
    a2 = (uint*)bp2->data;
    for (k = 0; k < NINDIRECT; k++) // 遍历 256 个数据块
        if (a2[k]) bfree(ip->dev, a2[k]);
    brelse(bp2);
    bfree(ip->dev, a[j]); // 释放一级索引块
}
}
brelse(bp);
bfree(ip->dev, ip->addrs[NDIRECT + 1]); // 释放二级索引块
ip->addrs[NDIRECT + 1] = 0;
}

```

5. 运行 `bigfile` 验证大文件创建，运行 `usertests` 确保正确性。



```

xv6 kernel is booting
init: starting sh
$ bigfile
.....
wrote 65803 blocks
reading bigfile
.....
bigfile done; ok

```

3) 实验中遇到的问题和解决办法

- **问题 1: NDIRECT 从 12 改为 11 后 addrs 数组大小不变**

- **解决办法:** 原来 `addrs[NDIRECT+1] = addrs[13]` (12 直接+1 间接)，改为 `addrs[NDIRECT+2] = addrs[13]` (11 直接+1 间接+1 二级间接)。数组总大小恰好不变，但 `struct dinode` 和 `struct inode` 中的字段语义改变：`addrs[0..10]` 直接块、`addrs[11]` 一级间接、`addrs[12]` 二级间接。需同时修改 `fs.h` 和 `file.h` 保持一致，否则磁盘 `inode` 和内存 `inode` 布局不匹配。

- **问题 2: 二级间接块的索引计算**

- **解决办法:** 块号 `bn` 减去 `NDIRECT` 后先判断一级间接区域，再减去 `NINDIRECT` 进入二级间接区域。二级区域内用 `bn_double = bn - NINDIRECT` (注意 `bn` 已经减过 `NDIRECT`)，一级索引 `idx1 = bn_double / NINDIRECT`，二级索引 `idx2 = bn_double % NINDIRECT`。三级映射：

`addrs[12]` → 二级索引块 → `a[idx1]` → 一级索引块 → `a2[idx2]` → 数据块。

- **问题 3: `itrunc()` 释放二级间接块时需要两层嵌套循环**

- **解决办法:** 外层 `for (j=0; j<NINDIRECT; j++)` 遍历二级索引块中的 256 个一级索引指针, 对每个非零的一级索引块 `a[j]`, 内层 `for (k=0; k<NINDIRECT; k++)` 遍历其 256 个数据块指针 `a2[k]` 逐一 `bfree`。释放顺序从内到外: 先释放数据块, 再释放一级索引块 `bfree(a[j])`, 最后释放二级索引块 `bfree(addrs[NINDIRECT+1])`。漏掉任何一层会导致磁盘块泄漏。

- **问题 4: `bmap()` 中二级间接块的三层按需分配**

- **解决办法:** 当目标块尚未分配时, `bmap` 需要逐层判断并分配: 先检查 `addrs[12]` (二级索引块) 是否存在, 不存在则 `balloc`; 再 `bread` 二级索引块, 检查 `a[idx1]` (一级索引块) 是否存在, 不存在则 `balloc` 并 `log_write`; 最后 `bread` 一级索引块, 检查 `a2[idx2]` (数据块), 不存在则 `balloc` 并 `log_write`。每层分配后都要 `log_write(bp)` 将修改写回日志。

4) 实验心得

通过本次实验, 我深入理解了文件系统中多级索引结构的设计思想。`xv6` 原本的单层间接块结构限制了文件大小, 引入二级间接块后, 文件容量得到了数量级的提升。这个设计权衡在实际文件系统中很常见: `ext4` 使用了类似的直接块+一级间接+二级间接+三级间接的组合策略, 而现代文件系统 (如 `XFS`、`Btrfs`) 则采用更灵活的 `B+` 树索引来支持超大文件。

在实现过程中, 最容易出错的地方是索引计算和块释放逻辑。三级地址映射需要仔细计算每一层的索引位置, 稍有不慎就会导致数据错乱。而释放操作需要从最内层向外层逐级释放, 确保所有已分配的块都被正确回收, 避免磁盘空间泄漏。

此外, 我还体会到修改文件系统底层结构的高风险性。`inode` 布局的改变意味着磁盘上的 `inode` 格式发生了变化, 必须确保 `mkfs` 工具创建文件系统时使用正确的 `inode` 结构, 否则会导致文件系统无法挂载。修改后需要重新 `make clean` 并重新运行 `make` 来重新编译 `mkfs`。

2. Symbolic links (moderate)

1) 实验目的

本次实验的目的是在 xv6 文件系统中添加符号链接 (symlink) 支持。符号链接是一种特殊类型的文件，其内容是另一个文件或目录的路径名。当进程访问符号链接时，内核会自动解析链接指向的目标文件。需要实现 `symlink()` 系统调用，并在路径解析过程中支持符号链接的自动解析，使得用户程序可以透明地使用符号链接访问目标文件。

2) 实验步骤

1. 在 `kernel/stat.h` 中定义 `T_SYMLINK` 类型和 `O_NOFOLLOW` 标志: `#define T_SYMLINK 4` // 符号链接文件类型在 `kernel/fcntl.h` 中添加 `#define O_NOFOLLOW 0x0100`。
2. 注册系统调用: 在 `user/user.h`、`usys.pl`、`kernel/syscall.h`、`kernel/syscall.c` 中添加 `symlink`。
3. 在 `kernel/sysfile.c` 中实现 `sys_symlink()`:

```
uint64 sys_symlink(void) {
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;
    if (argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
        return -1;
    begin_op();
    ip = create(path, T_SYMLINK, 0, 0); // 用 create 创建符号链接 inode
    if (ip == 0) { end_op(); return -1; }
    // 将 target 路径写入 inode 数据块
    if (writei(ip, 0, (uint64)target, 0, strlen(target) + 1) < 0) {
        iunlockput(ip); end_op(); return -1;
    }
    iunlockput(ip);
    end_op();
    return 0;
}
```
4. 在 `kernel/sysfile.c` 的 `sys_open()` 中添加符号链接跟随逻辑: `if (ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)) {`

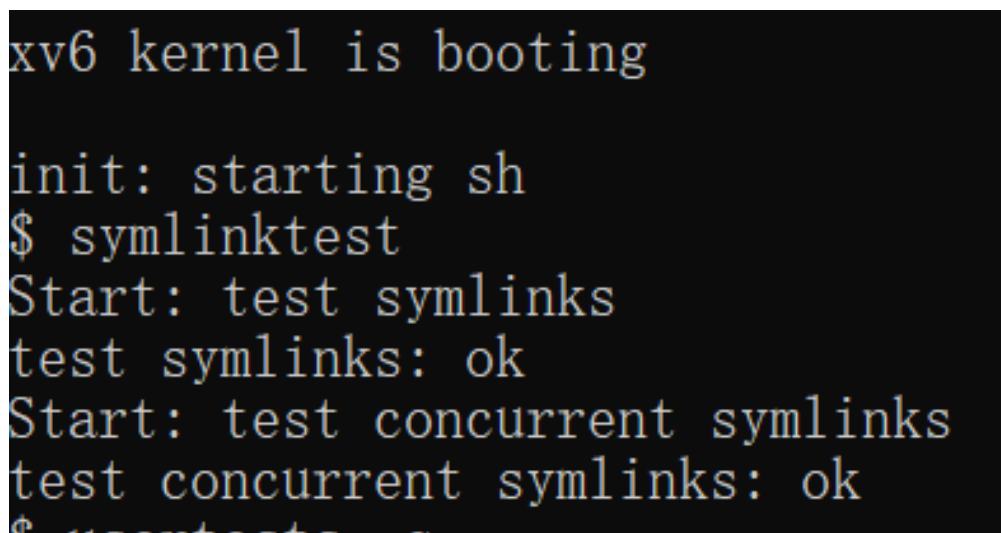
```
    int depth = 0;
    char link_target[MAXPATH];
    while (ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)) {
        if (++depth > 10) {           // 超过 10 层，防环路
```

```

    iunlockput(ip); end_op(); return -1;
}
memset(link_target, 0, MAXPATH);
if (readi(ip, 0, (uint64)link_target, 0, MAXPATH) <= 0) {
    iunlockput(ip); end_op(); return -1;
}
iunlockput(ip);          // 释放当前 inode
if ((ip = namei(link_target)) == 0) { // 解析目标路径
    end_op(); return -1;
}
ilock(ip);
if (ip->type == T_DIR && omode != O_RDONLY) {
    iunlockput(ip); end_op(); return -1; // 目录不可写打开
}
}
}
}

```

5. 运行 `symlinktest` 验证功能，运行 `usertests` 确保正确性。



```

xv6 kernel is booting
init: starting sh
$ symlinktest
Start: test symlinks
test symlinks: ok
Start: test concurrent symlinks
test concurrent symlinks: ok
$ usertests

```

3) 实验中遇到的问题 and 解决办法

- 问题 1: 符号链接跟随应该在 `namex()` 还是 `sys_open()` 中实现
 - 解决办法: 最初想在 `namex()` 中跟随符号链接, 但 `namex()` 是路径解析的底层函数, 被 `namei()` 和 `nameiparent()` 调用, 在其中跟随符号链接会导致所有路径操作都跟随链接, 包括 `stat()` 等。最终只在 `sys_open()` 中实现跟随——`sys_open` 通过 `namei()` 获取 `inode` 后, 检查 `ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)`, 用 `while` 循环跟随。 `stat()` 不做跟随, 直接返回符号链接本身的信息。
- 问题 2: 创建符号链接用 `ialloc()` 还是 `create()`

- **解决办法**：用 `create(path, T_SYMLINK, 0, 0)` 而非直接 `ialloc()`。`create()` 封装了 `ialloc()` + 在父目录中添加目录项 + `iunlockput` 逻辑，与 `sys_open` 创建普通文件的流程一致。手动 `ialloc` 还需要自己处理父目录链接，容易出错。`create` 返回已锁定的 `inode`，直接用 `writel` 写入 `target` 路径即可。

- **问题 3：跟随符号链接时的循环检测**

- **解决办法**：`sys_open` 中用 `while` 循环跟随符号链接，维护 `depth` 计数器，每次 `++depth`，超过 10 就返回 -1。用循环而非递归——因为递归需要保存中间状态（已解析的路径前缀），而 `sys_open` 中跟随是整体替换路径：每次 `readl` 读出 `target`，`iunlockput` 释放当前 `inode`，`namei(link_target)` 重新从根目录解析整个路径。

- **问题 4：O_NOFOLLOW 标志的处理**

- **解决办法**：在 `sys_open` 的跟随条件中加入 `!(omode & O_NOFOLLOW)`。当用户设置 `O_NOFOLLOW` 时，即使 `inode` 是 `T_SYMLINK` 也不跟随，直接打开符号链接文件本身（用于 `lstat` 等场景）。未设置时默认跟随到底，直到目标不是符号链接或达到深度上限。

- **问题 5：readl 读取链接目标时 inode 锁的状态**

- **解决办法**：`sys_open` 中已 `ilock(ip)`，`readl` 要求调用者持锁，所以直接调用 `readl(ip, 0, (uint64)link_target, 0, MAXPATH)` 读取。读取后需要 `iunlockput(ip)` 释放当前符号链接 `inode`，再 `namei(link_target)` 获取目标 `inode` 并 `ilock`。注意不能在持锁状态下调用 `namei`——`namei` 内部可能需要加锁其他 `inode`，导致死锁。

4) 实验心得

符号链接是文件系统中非常实用的功能，它提供了一种灵活的文件引用机制。通过符号链接，用户可以用不同的路径名访问同一个文件，类似于 Windows 中的快捷方式或 Linux 中的软链接。

实现符号链接的关键在于**路径解析的透明性**。当用户访问符号链接时，内核需要自动将其重定向到目标文件，这个过程对用户程序是不可见的。核心修改点在 `namex()` 函数——它是路径解析的内部实现，被 `namei()` 和 `nameiparent()` 调用。在 `namex()` 中检测到符号链接后，需要读取链接内容并重新调用 `namei()` 解析新路径。

另一个重要的设计考量是**符号链接的存储位置**。符号链接的内容存储在 `inode` 的数据块中，这意味着短路径的符号链接可以全部存储在直接块中（最多 11 个直接块），只

有当链接目标路径很长时才需要分配间接块。这种优化对于大多数实际场景（符号链接路径通常较短）非常有效。

此外，符号链接的环路检测也是一个重要的安全考量。恶意用户可能创建循环符号链接导致系统崩溃，因此必须设置递归深度限制。这让我联想到实际操作系统中的类似防护机制，如 Linux 内核中符号链接解析的最大深度限制。

Lab: mmap

1. mmap (moderate)

1) 实验目的

本次实验的目的是在 xv6 中实现 `mmap` 和 `munmap` 系统调用，将文件映射到进程的虚拟地址空间。通过实现虚拟内存区域（VMA）管理、惰性页面分配、页面缺失处理和文件写回机制，深入理解操作系统内存管理与文件系统的交互。具体目标包括：

- 实现 `mmap()` 系统调用，将文件内容映射到用户进程地址空间
- 实现 `munmap()` 系统调用，取消映射并写回脏页
- 支持 `MAP_SHARED`（共享映射，修改写回文件）和 `MAP_PRIVATE`（私有映射，修改不写回）
- 支持 `PROT_READ`、`PROT_WRITE` 等内存保护标志
- 实现惰性分配（lazy allocation），仅在页面缺失时分配物理页面
- 正确处理 `fork()` 后子进程对映射的继承
- 支持部分取消映射（prefix/suffix unmap）

2) 实验步骤

1. 在 `kernel/proc.h` 中定义 VMA 数据结构：`#define NVMA 16`

```

struct vma {
    int used;        // VMA 是否在使用
    uint64 addr;     // 映射起始虚拟地址
    uint64 len;      // 映射长度 (字节)
    int prot;        // PROT_READ / PROT_WRITE / PROT_EXEC
    int flags;       // MAP_SHARED / MAP_PRIVATE
    struct file *f;  // 被映射的文件
    uint offset;     // 文件偏移量 (本实验固定为 0)
}; 在 struct proc 中添加 struct vma vmas[NVMA] 和 uint64 mmap_top 字段。

```

2. 在 `kernel/sysfile.c` 中实现 `sys_mmap()`: `uint64 sys_mmap(void)` {

```

// argaddr(0..5) 解析 addr, len, prot, flags, fd, offset
if ((flags & MAP_SHARED) && (prot & PROT_WRITE) && !f->writable)
    return -1; // 共享+写需要文件可写
// 查找空闲 VMA 槽位
for (int i = 0; i < NVMA; i++)
    if (p->vmas[i].used == 0) { v = &p->vmas[i]; break; }
uint64 alen = PGROUNDUP(len);
if (alen > p->mmap_top - p->sz) return -1; // 空间不足
uint64 va = p->mmap_top - alen; // 向下分配
// 检查与已有 VMA 重叠
for (int i = 0; i < NVMA; i++) { ... overlap check ... }
v->used = 1; v->addr = va; v->len = len;
v->prot = prot; v->flags = flags;
v->f = filedup(f); // 增加文件引用计数
v->offset = 0;
p->mmap_top = va; // 更新下一次分配起点
return va;
}

```

3. 在 `kernel/proc.c` 中实现 `find_vma()`: `struct vma *find_vma(struct proc *p, uint64 va)` {

```

for (int i = 0; i < NVMA; i++)
    if (p->vmas[i].used && va >= p->vmas[i].addr
        && va < p->vmas[i].addr + p->vmas[i].len)
        return &p->vmas[i];
return 0;
}

```

4. 在 `kernel/proc.c` 中实现 `mmap_page_fault()`: `int mmap_page_fault(struct proc *p, struct vma *v, uint64 va)` {

```

int scause = r_scause();
if (scause == 15 && !(v->prot & PROT_WRITE)) return -1; // 写但不可写
if (scause == 13 && !(v->prot & PROT_READ)) return -1; // 读但不可读
va = PGROUNDDOWN(va);
pte_t *pte = walk(p->pagetable, va, 0);
if (pte && (*pte & PTE_V)) return 0; // 已映射, 无需处理

```



```

char *mem = kalloc();
if (mem == 0) return -1;
memset(mem, 0, PGSIZE);
uint64 fileoff = v->offset + (va - v->addr);
ilock(v->f->ip);
readi(v->f->ip, 0, (uint64)mem, fileoff, PGSIZE); // 读文件内容到物理页
iunlock(v->f->ip);
int perm = PTE_U;
if (v->prot & PROT_READ) perm |= PTE_R;
if (v->prot & PROT_WRITE) perm |= PTE_W;
if (v->prot & PROT_EXEC) perm |= PTE_X;
if (mappages(p->pagetable, va, PGSIZE, (uint64)mem, perm) != 0) {
    kfree(mem); return -1;
}
return 0;
}

```

5. 在 **kernel/sysfile.c** 中实现 **sys_munmap()**：解析 **addr** 和 **len**，调用 **vma_unmap()** 完成实际工作。

```

6. 在 kernel/proc.c 中实现 vma_unmap()：void vma_unmap(struct proc *p, struct
vma *v, uint64 addr, uint64 len) {
    // 裁剪请求范围到 VMA 边界
    if (addr < v->addr) { len -= (v->addr - addr); addr = v->addr; }
    if (addr + len > v->addr + v->len) len = v->addr + v->len - addr;
    uint64 unmap_start = PGROUNDDOWN(addr);
    uint64 unmap_end = PGROUNDUP(addr + len);
    // MAP_SHARED 写回脏页
    if (v->flags & MAP_SHARED) {
        for (uint64 a = unmap_start; a < unmap_end; a += PGSIZE) {
            uint64 pa = walkaddr(p->pagetable, a);
            if (pa == 0) continue; // 页面未分配，跳过
            uint64 fileoff = v->offset + (a - v->addr);
            begin_op();
            ilock(v->f->ip);
            writei(v->f->ip, 0, pa, fileoff, PGSIZE); // 写回文件
            iunlock(v->f->ip);
            end_op();
        }
    }
    uvmunmap(p->pagetable, unmap_start, (unmap_end - unmap_start)/PGSIZE, 1);
    // 调整 VMA：前缀/后缀/全部
    if (addr == v->addr && len == v->len) { v->used = 0; fileclose(v->f); }
    else if (addr == v->addr) { v->addr += len; v->len -= len; } // 前缀
    else { v->len -= len; } // 后缀
}

```

7. 在 **kernel/trap.c** 的 **usertrap()** 中添加页面缺失处理：else if (r_scause() == 15 || r_scause() == 13) {
 uint64 va = r_stval();


```

struct vma *v = find_vma(p, va);
if (v != 0) {
    if (mmap_page_fault(p, v, va) < 0)
        setkilled(p);          // 非法访问, 杀死进程
    } else if (vmfault(p->pagetable, va, (r_scause() == 13) ? 1 : 0) == 0) {
        setkilled(p);          // 非 mmap 区域的意外页错误
    }
}

```

8. 在 `kernel/proc.c` 的 `kfork()` 中复制 VMA 表: `for (int i = 0; i < NVMA; i++) {`

```

    if (p->vmass[i].used) {
        np->vmass[i] = p->vmass[i];    // 复制 VMA 结构
        filedup(np->vmass[i].f);      // 增加文件引用计数
    }
}
np->mmap_top = p->mmap_top;    // 继承地址空间布局

```

9. 在 `kernel/proc.c` 的 `kexit()` 中清理所有映射: `for (int i = 0; i < NVMA; i++) {`

```

    if (p->vmass[i].used)
        vma_unmap(p, &p->vmass[i], p->vmass[i].addr, p->vmass[i].len);
}

```

10. 运行 `mmaptest` 验证。



fht@LAPTOP-1TQ69S4D: ~/xv6-labs-2025

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ mmaptest
test basic mmap
test basic mmap: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
test fork: OK
test munmap prevents access
usertrap(): unexpected scause 0xd pid=5
             sepc=0xa2c stval=0x3ffffee000
usertrap(): unexpected scause 0xd pid=6
             sepc=0xab4 stval=0x3ffffed000
test munmap prevents access: OK
test writes to read-only mapped memory
usertrap(): mmap fault scause 0xf pid=7
             sepc=0xbfa stval=0x3ffffeb000
test writes to read-only mapped memory: OK
mmaptest: all tests succeeded
$
```

3) 实验中遇到的问题和解决办法

- 问题 1: mmap 区域的虚拟地址空间布局

- 解决办法: 采用从高地址向低地址增长的策略, `mmap_top` 在 `allocproc` 中初始化为 `PGROUNDDOWN(TRAPFRAME)`, 每次 `mmap` 向下分配 `uint64 va = p->mmap_top - PGROUNDUP(len)`。需要检查空间是否足够: `if (alen > p->mmap_top - p->sz) return -1`, 确保不会侵入堆区。分配后更新 `p->mmap_top = va`。

- 问题 2: 页面缺失时文件内容的读取

- 解决办法: 在 `mmap_page_fault()` 中, 计算文件偏移量 `fileoff = v->offset + (va - v->addr)`, `ilock(v->f->ip)` 锁定 `inode`, `readi(v->f->ip, 0, (uint64)mem, fileoff, PGSIZE)` 从文件读取到物理页, `iunlock(v->f->ip)` 释放锁。 `readi` 不需要文件结构体, 直接操作 `inode`, 比 `fileread` 更底层。读取后用 `mappages` 建立映射, 权限由 `perm = PTE_U | (PROT_READ?PTE_R:0) | (PROT_WRITE?PTE_W:0) | (PROT_EXEC?PTE_X:0)` 构建。

- 问题 3: MAP_SHARED 脏页写回时机与日志层

- 解决办法: 在 `vma_unmap()` 中, 对 `MAP_SHARED` 的映射, 遍历范围内每个有效页面, 用 `walkaddr` 获取物理地址 `pa`, 计算 `fileoff`, 然后 `begin_op()` $\rightarrow$ `ilock(v->f->ip)` $\rightarrow$ `wrotei(v->f->ip, 0, pa, fileoff, PGSIZE)` $\rightarrow$ `iunlock` $\rightarrow$ `end_op()`。 `begin_op()/end_op()` 是日志层事务包装——`wrotei` 修改磁盘块必须包裹在日志事务中, 否则系统崩溃会导致文件系统不一致。本实验不检查 `PTE_D` (脏位), 对所有已分配页面都写回。

- 问题 4: fork() 后子进程的 VMA 处理

- 解决办法: `kfork()` 中 `np->vmass[i] = p->vmass[i]` 复制 VMA 结构体, `filedup(np->vmass[i].f)` 增加文件引用计数, `np->mmap_top = p->mmap_top` 继承地址空间布局。子进程的物理页面在页面缺失时才分配 (惰性策略), 父子进程独立触发 page fault、独立分配页面。子进程 `munmap` 或 `exit` 时通过 `vma_unmap` 独立清理。

- 问题 5: 部分取消映射 (prefix/suffix unmap) 的实现

- 解决办法: `vma_unmap()` 先将请求范围裁剪到 VMA 边界内, 再判断类型:
 - 全部: `addr == v->addr && len == v->len  $\rightarrow$  v->used = 0; fclose(v->f)`
 - 前缀: `addr == v->addr  $\rightarrow$  v->addr += len; v->len -= len` (起始地址前移)
 - 后缀: `addr + len == v->addr + v->len  $\rightarrow$  v->len -= len` (仅缩短长度)注意: 本实验 `v->offset` 固定为 0, 不需要调整 `offset`。页面释放在

`uvmunmap(p->pagetable, unmap_start, npages, 1)` 释放物理内存。

● 问题 6：页面缺失时的权限检查

- 解决办法：在 `mmap_page_fault()` 中通过 `r_scause()` 判断异常类型：
 - `scause == 15` (store/AMO 异常) 但 `!(v->prot & PROT_WRITE)` → 返回 -1
 - `scause == 13` (load 异常) 但 `!(v->prot & PROT_READ)` → 返回 -1
 - 返回 -1 后在 `usertrap()` 中 `setkilled(p)` 杀死进程。同时检查 `walk(p->pagetable, va, 0)` 返回的 PTE 是否已有 `PTE_V`——已映射则直接返回 0，避免重复分配。

● 问题 7：mmap 后立即 close(fd) 映射仍有效

- 解决办法：`sys_mmap()` 中 `v->f = filedup(f)` 增加文件引用计数，`filedup` 将 `f->ref++`。即使进程随后 `close(fd)` 调用 `fileclose` 将 `f->ref--`，只要 VMA 持有的引用还在，`struct file` 就不会被释放。直到 `vma_unmap` 中 `fileclose(v->f)` 才真正释放。这保证了 `mmap` 映射不依赖于 `fd` 是否打开。

4) 实验心得

通过本次实验，我深入理解了内存映射文件（mmap）的实现原理。关键收获包括：

1. **惰性分配策略**：mmap 采用与 `sbrk` 相同的惰性分配思想，在 `mmap` 调用时只记录 VMA 信息，实际物理页面在页面缺失时才分配。这种策略的优势在于：如果映射的部分页面从未被访问，则永远不会分配物理内存，节省了内存开销。
2. **VMA 抽象**：VMA 结构将进程地址空间中的不同内存区域统一管理。每个 VMA 记录了地址范围、权限、关联文件等信息。这种抽象使得 `mmap` 的实现简洁清晰，也为后续实现更复杂的内存管理（如地址空间布局随机化）提供了基础。
3. **共享映射与私有映射**：`MAP_SHARED` 实现了内存与文件的双向同步——写入映射区域会写回文件，其他映射同一文件的进程可以看到修改。`MAP_PRIVATE` 则采用写时复制（COW）策略，修改只影响本进程的私有副本。这种设计体现了操作系统在效率（共享内存）和安全（隔离性）之间的权衡。
4. **地址空间布局**：`mmap` 区域位于堆和栈之间，这种布局在现代操作系统中很常见。`mmap` 区域向下增长，堆向上增长，栈位于高地址。这种布局为动态内存分配和文件映射提供了灵活的地址空间管理。
5. **进程生命周期管理**：`mmap` 涉及的资源（物理页面、文件引用）需要在进程生命周期的各个阶段正确管理——`fork` 时继承、`munmap` 时部分释放、`exit` 时全部清理。这让

我深刻理解了操作系统资源管理的复杂性。